STARLET Utility Routines

Programmer's Guide

STARLET-PG-0171

STARLET Version 1.7.1 *** 31-OCT-2026

Copyright © 2026 StarLet Squad and Ruslan R. Laishev.

StarLet is free software, distributed under the Apache License, Version 2.0. It comes with ABSOLUTELY NO WARRANTY.

Contents

Preface **5**
Intended Audience . . . 5
Document Structure . . . 6
Related Documents . . . 7
Acknowledgments . . . 8
Trademarks . . . 9
Conventions . . . 10

Chapter 1. Concept **11**
1.1 Why StarLet Exists . . . 11
1.2 What StarLet Is . . . 12
1.3 What StarLet Is Not . . . 13
1.4 Why These Routines Deserve to Exist . . . 14
1.5 Decisions and Why . . . 16
1.6 Who Uses It . . . 17
1.7 A Course of Study . . . 18

Chapter 2. Getting Started **19**
2.1 What You Need . . . 19
2.2 Building . . . 20
2.3 Installing as a Package . . . 21
2.4 Without CMake . . . 22
2.5 The First Program . . . 23
2.6 What the Program Did . . . 24

Chapter 3. Condition Values and Messages **25**
3.1 The Layout of a Condition Value . . . 25
3.2 Severities and the Low Bit . . . 26
3.3 Codes the Library Returns . . . 27
3.4 Defining a Facility and Its Messages . . . 28
3.5 The Catalogue . . . 29
3.6 Signalling a Message . . . 30
3.7 Getting the Text Without Printing . . . 31

Chapter 4. Logging **32**
4.1 The Routines at a Glance . . . 32
4.2 The Record . . . 33
4.3 `$LOG_FAO` . . . 34
4.4 Where the Log Goes . . . 35
4.5 Rotation . . . 36
4.6 Syslog and Logcat . . . 37
4.7 Tracing and Dumping . . . 38
4.8 Records into a Buffer . . . 39

Chapter 5. FAO, the Formatter **40**
5.1 Why FAO . . . 40

5.2 Parameters . . . 41
5.3 The Directives . . . 42
5.4 A Tour of Real Output . . . 44
5.5 FAO in Messages and Logs . . . 45
5.6 Gotchas . . . 46

Chapter 6. Strings and Data **47**
6.1 The Counted String . . . 47
6.2 Moving and Searching . . . 49
6.3 Cleaning Up Text . . . 50
6.4 Wildcards . . . 51
6.5 Base64 . . . 52
6.6 Hexadecimal and UUID Text . . . 53
6.7 CRC-32 . . . 54
6.8 Item Lists . . . 55
6.9 Small Things . . . 56

Chapter 7. Keywords and Parameters **57**
7.1 Looking up a Keyword . . . 57
7.2 Values to Words . . . 59
7.3 Options of the Command Line and of a File . . . 60

Chapter 8. The DCL-Style Command Line **63**
8.1 The Idea . . . 63
8.2 The Tables . . . 64
8.3 A Complete Program . . . 65
8.4 The Routines . . . 68
8.5 Rules of the Parser . . . 69
8.6 Where It Differs from DCL . . . 70

Chapter 9. Queues and Locks **71**
9.1 The Heritage . . . 71
9.2 The Types . . . 72
9.3 The Operations . . . 73
9.4 Atomics . . . 74

Chapter 10. Time **75**
10.1 Binary Time Here . . . 75
10.2 The Text . . . 76
10.3 Arithmetic . . . 77
10.4 The Other Time Routines . . . 78

Chapter 11. The TLV Protocol **79**
11.1 The Idea . . . 79
11.2 The Routines . . . 80
11.3 Cautions . . . 82

Chapter 12. Portability **83**
12.1 What Is Tested Where . . . 83
12.2 The Rules the Code Follows . . . 84

Chapter 13. The House Style **85**
13.1 Naming . . . 86
13.2 Formatting . . . 87
13.3 Identification and Revision . . . 88
13.4 The Modification History and the Functional Description . . . 89
13.5 Condition Values, Not Prints . . . 90

Chapter 14. The Tests **91**
14.1 What Is There . . . 91

Appendix A. Quick Reference Card **94**

Appendix B. History of the Library **96**

Appendix C. Known Limitations: What Is Still True **99**

Appendix D. Changes in Versions 1.7.0 and 1.7.1 **101**
Changes in Version 1.7.1 101
Changes in Version 1.7.0 103

Preface

Intended Audience

This manual is for the programmer who wants to use StarLet in a program of his own, and for the one who has to change StarLet itself: to add a directive, a message, a qualifier or a routine, to correct a defect, or to satisfy himself that a change is safe. It assumes C, a Linux shell and a reading acquaintance with CMake. It does not assume OpenVMS. A reader who knows OpenVMS will, however, feel at home in a few minutes, and will find that some of the names are old friends.

The student will find here a small library that can be read from the first line to the last in a few evenings, and that explains why each part is made the way it is. The veteran will find the things he remembers - the low bit of a condition value, the counted string, the control string of `$FAO`, the qualifier with its `/NO` form - and, we hope, will forgive the places where a Linux program had to do it differently.

The *STARLET Reference Manual* lists every routine, one by one. This manual teaches how to use them together, and neither repeats the other.

Document Structure

Part	Contents
Chapter 1	Why StarLet exists, what it is and is not, why its routines are worth having, a course of study
Chapter 2	Getting started: building, installing, and the first complete program
Chapter 3	Condition values and the message catalogue
Chapter 4	Logging: records, files, rotation, trace and hex dumps
Chapter 5	FAO, the formatter: every directive, with examples
Chapter 6	Strings and data: counted strings, wildcards, base64, hex, CRC-32, item lists
Chapter 7	Keywords and parameters: lookup tables, command-line options, configuration files
Chapter 8	The DCL-style command line: verbs, parameters, qualifiers, help
Chapter 9	Queues, spin locks and atomics
Chapter 10	Time: binary time, delta times, the ASCII forms
Chapter 11	The TLV protocol
Chapter 12	Portability: Linux, ARMv8, Windows, Android, musl
Chapter 13	The house style of the sources
Chapter 14	The tests and what each of them guards
Appendix A	Quick reference card
Appendix B	History of the library
Appendix C	Known limitations: what is still true
Appendix D	Changes in Versions 1.7.0 and 1.7.1

Related Documents

- *STARLET Reference Manual* (order number STARLET-RM-0171): every routine and macro, in the layout of a run-time library reference.
- The sources themselves. Each module opens with its identification and its modification history, and every routine carries a `FUNCTIONAL DESCRIPTION` block. Where this manual and that block disagree, the block - and then the code - is right, and this manual is to be corrected.

Acknowledgments

The authors thank Claude, the AI model of Anthropic, which read the sources, wrote and ran the tests and the examples, and went through this manual together with them.

The authors thank their comrades-in-arms of the NaPi Lab community, who paid for the subscription to Claude, so that an engineer of almost pensionable age could go on with this work.

And the authors thank the engineers of Digital Equipment Corporation, whose habits of work are the whole reason this library looks the way it does. Most of them will never read these lines. We hope they would have recognized the result.

Trademarks

OpenVMS, VAX, Alpha, Digital and DEC are trademarks of Hewlett Packard Enterprise Development LP or of VMS Software, Inc. Linux is the registered trademark of Linus Torvalds in the U.S. and other countries. Windows is a trademark of the Microsoft group of companies. Android is a trademark of Google LLC. Arm is a trademark of Arm Limited. Intel is a trademark of Intel Corporation. GLib is a part of the GNOME project. APR is a project of the Apache Software Foundation. MinGW-w64 and musl are free software projects of their respective authors. Claude is a trademark of Anthropic, PBC. All other names of products and companies are trademarks of their respective owners.

StarLet is an independent work. It is not affiliated with, endorsed or supported by any of the owners named here; their names are used only to say what StarLet resembles or runs on.

Conventions

Line numbers are not quoted: they go stale. A routine is named instead.

Convention	Meaning
`monospace`	A routine, macro, constant, file or fragment of code
`__util$name`	A routine of the library; a `$` in a name is plain text, as it was on OpenVMS
`$NAME`	A macro of the library
`STS$K_...`	A constant: a severity, or a completion code
`!AZ`, `!UL`	A directive of the FAO control string (Chapter 5)
status	A condition value, the integer a routine returns; its low bit tells success from failure
`[x]` in a synopsis	An optional argument

Code examples in this manual were compiled and run against the sources of StarLet 1.7.1. The output shown is the output they gave, except that the date, the time and the thread number of a log record are typical and will differ on your machine.

Chapter 1. Concept

1.1 Why StarLet Exists

Programs are written by people, and people forget. A function returns -1, and the caller never looks. A message is formatted into a buffer that was a little too small. A log line says “error” and nothing else, and three years later nobody remembers which error. A command line is parsed by a hand-made loop that accepts `-v` but not `-verbose`, and the next program of the same author parses it in another way.

The engineers of Digital Equipment Corporation met these problems early, and answered them not with a single clever tool but with a habit. Every routine returns a condition value, and the low bit says whether it worked. Every message has a facility, a number and a severity, and its text lives in a table, not in the middle of a function. Every string has its length with it. Every command is a verb with parameters and qualifiers, written down in a table, so that the parsing is done by the system and not by the author’s mood that day. This habit is why OpenVMS programs of 1985 still run, and still tell you what is wrong with them.

StarLet carries that habit to Linux, and from there to ARMv8 boards, to Windows and to Android. It is a small library of C11. It is not an emulation of OpenVMS, and it does not pretend to be one. It is the *shape* of the OpenVMS run-time library, made to work on the machines that are on our desks now, so that a program written today can be written the way a good program was written then.

1.2 What StarLet Is

StarLet is one static library, `libstarlet.a`, made of four modules.

Module	Prefix	What it gives
`utility_routines`	`__util$`, `$LOG`	Logging; the message catalogue; condition values; counted strings; queues with spin locks; options of the command line and of a configuration file; keyword lookup; base64; CRC-32; hex and UUID text; wildcard matching; and more
`starlet`	`__util$`	The FAO formatter, `__util$fao` and `__util$faol`; binary time, `__util$bintim`, `__util$asctim`, `__util$add_times`, `__util$sub_times`
`cli_routines`	`__cli$`	A DCL-style command parser: verb, parameters, qualifiers, typed values, dispatch, help
`avproto`	`__tlv$`	A compact tag-length-value binary protocol

The name needs a word. On OpenVMS, `STARLET` is the library in which the system services - `SYS$FAO`, `SYS$BINTIM`, `SYS$ASCTIM` and their relations - are declared. The module `starlet` of this package holds the portable counterparts of a few of them, and the whole package took the name from it.

The libraries of OpenVMS are the parents of the ideas here, and their names show up as the *places to look* when a routine seems to have no reason: `SYS$FAO`, `SYS$PUTMSG`, `SYS$BINTIM`, `SYS$ASCTIM`, `LIB$ADD_TIMES`, `LIB$SUB_TIMES`, `LIB$MOVC5`, `CLI$PRESENT`, `CLI$GET_VALUE`, and the `INSQUE` and `REMQUE` instructions. But each routine of StarLet is its own routine, with its own calling sequence, and Chapters 5 to 11 say plainly where it differs.

1.3 What StarLet Is Not

- **It is not a replacement for the C library.** It sits on top of it. Memory, files, sockets, threads, the clock - they all remain the business of libc and the kernel.
- **It is not a framework.** There is no main loop, no object system, no plug-in mechanism. You call a routine, it returns a condition value, and you go on.
- **It does not give you containers.** There is a queue, because the VAX had one. There is no hash table, no tree, no vector.
- **It is not a port of the OpenVMS run-time library.** It has no message vectors, no condition handlers, no `LIB$GET_INPUT`, no descriptors of the full OpenVMS family, no `SYS$QIO`. When a name of OpenVMS is borrowed, the routine behind it does what this manual says, and nothing more.
- **It does not do text encodings.** A string is a run of octets. What the octets mean is the program's affair.

1.4 Why These Routines Deserve to Exist

A fair reader will ask: the C library, GLib and the Apache Portable Runtime already exist, and are excellent. Why another one? It is the right question, and the answer is not “StarLet does more”. It does less. What it does is hold to one discipline from the first routine to the last, and the discipline is the product. Here is what it gets right, plainly, and where each point stops being true.

1. A failure cannot hide behind a convention that nobody wrote down. The routines that can fail return a condition value, and the test is always the same: `1 & status`. Success has the low bit set, failure does not. There is no table of “this one returns zero on success, that one returns -1, a third returns a pointer or NULL”. A student learns one test and can read any call. A veteran knows it as `SS$_NORMAL` - odd means well. *Where this stops:* not every routine can fail. `__util$crc32c` returns a checksum, `__util$pattern_match` a true or false, the hex helpers a length. And a library cannot force a caller to look: it can only make the right test the easy one. The FAO engine, for one, only returns its complaints and logs nothing, and `$LOG_FAO` does not look at them (Section 5.6).

2. One catalogue, one look for every diagnostic. A message is declared once, in a table: its facility, its number, its severity and its text. When it is signalled, the output is always in the form `%FAC-S-IDENT, text` - the form every OpenVMS user has read ten thousand times. The text is not glued into the code, so it can be corrected, reviewed and translated in one place, and the same condition value tells a program *which* thing went wrong without anybody parsing a sentence.

3. Lengths travel with the data. A counted string (`ASC`, the ASCIC of OpenVMS) carries its length in its first octet. Routines that fill a buffer take the buffer, its size, and a place to return the length: `__util$base64`, `__util$log2buf`, `__util$faol`, `__tlv$get`. Nobody has to guess where a buffer ends, and nobody has to call `strlen` again on a string whose length was known a line ago. A formatter that runs out of room truncates and *says so* in its return value; it does not write past the end - not even for a control string cut short in the middle of a directive, nor for a TLV record that claims more octets than the packet has. *Where this stops:* an `ASC` holds at most 255 octets, and some routines still accept zero-terminated strings, because C does.

4. One formatter for everything. `$LOG_FAO`, `$PUTMSG_FAO` and `__util$fao` all drive the same engine, with the same directives. A line written to a log, a message in the catalogue and a string formatted for a screen use the same `!UL`, `!AZ`, `!XL`. The engine knows things `printf` does not: `!%S` for a plural, `!%D` for a date, `!n<...!>` for a fixed field, and four directives for the objects of a network programmer’s day - `!%M` a MAC address, `!%I` an IPv4 address, `!%6` an IPv6 address in its RFC 5952 canonical form, `!%U` a UUID. It does not read the stack by accident: the macros count the parameters themselves.

5. A log record says who, when and where. Every record carries the date and time, the thread number, the facility and the severity. In a build without `NDEBUG` it also carries the module, the routine and the line of the call. The record is built in a buffer and goes out in a single `write()`, so two threads do not tear each other’s lines. *Where this stops:* the module, routine and line are a debug-build feature. A consumer built as Release gets the shorter form.

6. Small, static, with no dependencies. Four modules, one static library, about eleven thousand lines with their comments. The library needs the C library and nothing else (and Winsock on Windows). It is plain C11. You add it to a program with three lines of CMake, and you can read all of it.

7. It is tested, on more than one libc. Twenty-four test programs are run by CTest - twenty-five runs, since the CRC-32 is run twice - and a continuous-integration job builds them with glibc under AddressSanitizer and UndefinedBehaviorSanitizer, and again with musl. The hostile-input tests (a control string cut short, a TLV that lies about its length) allocate their buffers to the exact size, so a single stray octet is fatal under the sanitizer. Windows builds are run under Wine and, for the log rotation, on Windows 11. Chapter 14

says what each test guards.

8. A command line in the manner of DCL, in C. Verbs and sub-verbs, positional parameters, qualifiers with `/name=value` or `--name=value`, the negated form `/NOname`, value lists `/tag=(a,b,c)`, defaults, required items, typed values (a number, a file, a time, a MAC address, a UUID, a keyword), abbreviation of a name to its unique prefix, and a help screen made from the same tables. *Where this stops:* there is no CLD compiler - the tables are C initializers - a value must be attached with `=` (`/blocks 5` is refused, with a message that says so), and some corners are rougher than DCL's. Chapter 8 and Appendix C are honest about them.

None of this is a feat. It is a list of things that were *obviously right* in 1985, and that a modern C program can still have, if somebody takes the trouble to put them in one place. This is that place.

1.5 Decisions and Why

The low bit is the verdict. Success is odd, failure is even. `STS$K_SUCCESS` is 1 and `STS$K_INFO` is 3: both are “fine”. `STS$K_WARN` is 0, `STS$K_ERROR` is 2, `STS$K_FATAL` is 4: all “not fine”. Always test with `1 & status`, never with `== STS$K_SUCCESS`, because a routine may return a more specific success - and because the comparison fails on exactly the day somebody adds one. A test of the library, `test_status_convention`, exists to keep every code on the right side of this line.

A condition value is a number, and its text is elsewhere. A condition value packs a facility, a message number and a severity (Chapter 3). The words belong to a catalogue. A program decides by the number, a person reads the text. The two never get mixed.

Messages are signalled, not printed. A routine that finds a problem reports it with `$PUTMSG_FAO` and returns the same condition value, in one expression. The caller sees the diagnostic on the log and also has the status in its hand. This is how OpenVMS code does it with `LIB$SIGNAL`, and the expression `return $PUTMSG_FAO(...)` is the shortest honest form of it that Linux allows.

Output goes through one door. The library writes to a single descriptor, `g_logoutput` inside the module, set by `__util$deflog` or `__util$setlogfd`. There is no hook, no callback and no second channel (there was one once, and it was removed). A program that needs its standard output clean calls `__util$setlogfd(STDERR_FILENO)` once.

FAO, not `printf`, in our own sources. `printf` formats are checked by the compiler against types that in this library are nearly always the same: 32-bit numbers and strings. FAO adds what the system programmer needs and `printf` lacks, and has a single-source meaning on every platform. The house rule of our packages is that messages and logs go only through `$LOG_FAO` and `$PUTMSG_FAO` (Chapter 13). The older `$LOG`, with a `printf` format, is kept for those who need it, and is documented here.

Counted strings where the length matters. The `ASC` type is the ASCIC of OpenVMS: one length octet, then up to 255 octets of text. It is cheap to copy, safe to compare, and it can be built at compile time with `$ASCDECL`. The command tables of Chapter 8 are full of them.

Static, not shared. A library of this size does not earn a shared object. A static archive means no `LD_LIBRARY_PATH` surprises, a single file to copy, and a program that runs on a board with a bare root file system. The CMake package installs the archive, the headers and the usage requirement.

The engine does not depend on the library. `starlet.c`, which holds the FAO engine and the time routines, includes no `<stdio.h>`, `<string.h>` or `<ctype.h>`; the one routine it needs from the C library is `time()`, and a freestanding build can replace that too. It does its own digit conversion and its own calendar arithmetic. Even its diagnostics are off by default: its `$LOG` is an empty macro unless the includer supplies one. That makes it possible to lift the engine into a bootloader or a firmware image.

1.6 Who Uses It

The library came out of the real programs, and still serves them. Each of these takes its message catalogue, its logging, or its command line from StarLet:

- **BATCH** - a batch job subsystem: a daemon, a queue and a command-line client.
- **MSCP-Hub** - the MSCP disk server for OpenVMS nodes; its diagnostics go through `$PUTMSG_FAO`.
- **yaVTERM** - an encrypted terminal and file transfer.
- **AlphaEmu** - the Alpha system emulator; its console and boot keywords are looked up with `__util$lookup_key`.
- **SEARCH** - a search utility in the manner of `SEARCH` of OpenVMS; its message table is a worked example of a catalogue with an `inimsg` call.
- **yaTPU** - a text-processing utility in the manner of TPU.
- **HELP** - the help-library reader.

When this manual can be checked against a real program, those are the programs to open. SEARCH, in particular, is small and keeps its catalogue in one tidy table.

1.7 A Course of Study

The library is meant to be read. It is small, it is plain C, and every routine says in its header what it does. The chapters follow the order in which the ideas build on each other:

1. Chapter 2: run the first program, and see a message come out as `%HELLO-I-GREET`.
2. Chapter 3: add a message of your own; learn what the low bit does.
3. Chapter 5: play with FAO until the directives are in your fingers; everything else prints through it.
4. Chapters 4 and 6: write a small tool that reads a file, trims its lines and logs them.
5. Chapters 7 and 8: give it a command line with a verb and two qualifiers.
6. Chapters 9 to 11 when you need a queue, a time or a packet.
7. Chapter 13, and then `utility_routines.c` from the top. Its modification history is a short history of the craft: every entry says what was wrong and why it was changed.

And for the engineer who knew OpenVMS in its day: the names are the old ones, and so, mostly, are the ideas. Please read Appendices C and D too. A library that is honest about its corners, and about what it changed, is one you can trust.

Chapter 2. Getting Started

2.1 What You Need

A C11 compiler (GCC or Clang), CMake 3.10 or later, and a Linux, Windows (MinGW-w64) or Android build environment. The library is pure C; its `project()` line says `LANGUAGES C`, so a host with no C++ compiler configures without complaint.

2.2 Building

From the top of the sources:

```
cmake -B build
cmake --build build
ctest --test-dir build
```

The first line chooses a build directory. The second builds the static library, the CMake target **`starlet`**, and the test programs. The third runs the tests: twenty-five runs, all of which should pass.

If you name no build type, CMake builds **Release**. That is deliberate, and it has a consequence you should know now: a Release build defines `NDEBUG`, which compiles out the library's own `$TRACE` output and the trace of the command parser. If you want that tracing, ask for it:

```
cmake -B build-debug -DCMAKE_BUILD_TYPE=Debug
```

The Debug build also defines `_DEBUG`, `__TRACE__` and `DEBUG`.

2.3 Installing as a Package

```
cmake --install build --prefix /usr/local
```

This installs the archive `libstarlet.a`, the four public headers (`utility_routines.h`, `starlet.h`, `cli_routines.h`, `avproto.h`) and a CMake package under `lib/cmake/StarLet`. Another project then uses it without `add_subdirectory()` and without `FetchContent`:

```
cmake_minimum_required(VERSION 3.10)
project(hello LANGUAGES C)

find_package(StarLet REQUIRED)

add_executable(hello main.c)
target_link_libraries(hello PRIVATE StarLet::starlet)
```

If you installed to a private prefix, tell CMake where to look: `-DCMAKE_PREFIX_PATH=$HOME/stuff`. The imported target `StarLet::starlet` brings the include directory with it. On Windows it also brings `ws2_32`.

If you prefer to have the sources inside your own tree, `add_subdirectory(external/starlet)` gives you the same target under the same name, `StarLet::starlet`. In that case the package and CPack rules of StarLet are skipped, so a `cpack` in your tree builds *your* package.

A consumer's own build type matters. StarLet's `NDEBUG` is decided when the library is compiled, but the macros `$LOG`, `$LOG_FAO`, `$PUTMSG_FAO` and `$TRACE` are expanded in *your* code, so it is *your* translation unit that decides whether a record carries the module, the routine and the line.

2.4 Without CMake

Four source files, and a compiler:

```
cc -std=gnu11 -O2 -c utility_routines.c avproto.c cli_routines.c starlet.c
cc *.o -o your_app
```

Add `-DNDEBUG` for a quiet build. `sh tests/run.sh` builds and runs most of the tests the same way, honouring `CC` and `CFLAGS`.

2.5 The First Program

The following is a complete program. It declares a facility, two messages, registers the catalogue, and uses `$LOG_FAO` and `$PUTMSG_FAO`. Save it as `hello.c`.

```
#define __MODULE__  "HELLO"
#define __FAC__     "HELLO"

#include    <stdlib.h>
#include    "utility_routines.h"
#include    "starlet.h"

#define HELLO$K_FACNO   0x0200

#define HELLO$_GREET    ($DEFFAC(HELLO$K_FACNO) | $DEFMSG(1) | $DEFSEV(STS$K_INFO))
#define HELLO$_NOFILE   ($DEFFAC(HELLO$K_FACNO) | $DEFMSG(2) | $DEFSEV(STS$K_ERROR))

#define $HELLOREC(a_sts, a_txt) \
    { .sts = (int) (a_sts), .textl = (unsigned char) (sizeof(a_txt) - 1), .text = a_txt,
 ↪  .mnemonic = #a_sts }

static  EMSG_RECORD s_msgtab [] = {
    $HELLOREC(HELLO$_GREET, "Hello, !AZ!! You are visitor number !UL"),
    $HELLOREC(HELLO$_NOFILE, "cannot open !AZ, errno=!SL")
    };

static  EMSG_RECORD_DESC    s_msgdsc = {
    .msgnr      = (unsigned) $ARRSZ(s_msgtab),
    .facno      = HELLO$K_FACNO,
    .msgrec     = s_msgtab,
    .facname    = __FAC__
    };

int main    (int argc, char *argv[])
{
int l_sts;

    __util$inimsg(&s_msgdsc);

    $LOG_FAO(STS$K_INFO, "started with !UL argument!%S", argc - 1);

    l_sts = $PUTMSG_FAO(HELLO$_GREET, "Vasya", 7);
    l_sts = $PUTMSG_FAO(HELLO$_NOFILE, "/no/such", 2);

    return  (1 & l_sts) ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

Build and run it:

```
cc -std=gnu11 -I/usr/local/include hello.c /usr/local/lib/libstarlet.a -o hello
./hello a b
```

It prints three records, and exits with status 1, because the last message was an error:

```
31-10-2026 17:53:13.972 2702691 [HELLO\main:34] %HELLO-I:  started with 2 arguments
31-10-2026 17:53:13.972 2702691 [HELLO\main:36] %HELLO-I-GREET, Hello, Vasya! You are visitor number 7
31-10-2026 17:53:13.972 2702691 [HELLO\main:37] %HELLO-E-NOFILE, cannot open /no/such, errno=2
```

2.6 What the Program Did

The two defines. `__MODULE__` names the source module, shown in the `[module\routine:line]` part of a record. `__FAC__` is the facility name that follows the `%`. If you do not define `__FAC__`, the header supplies `"UTILS"` - the facility of the library itself - and your records claim to come from there. Define both before the include.

The include order. `utility_routines.h`, then `starlet.h`, as in the example. Since 1.7.0 the order of these two does not matter: `starlet.h` includes `utility_routines.h` itself, and the old fallback definitions of the severities, which broke the `enum` when the headers came the other way round, are gone. `starlet.h` is needed by `$LOG_FAO` and `$PUTMSG_FAO`, which cast every parameter to its type `fao_prm_t`. `cli_routines.h` and `avproto.h` still take their types from `utility_routines.h` and must come after it.

The record macro. `$HELLOREC` fills one `EMSG_RECORD`: the condition value, the length of the text (an ASCIC, as in OpenVMS), the text, and the mnemonic taken from `#a_sts`, the stringized name `"HELLO$_GREET"`. `__util$inimsg` trims it to the part after `$_`, so the record prints as `GREET`. The macro parameters are called `a_sts` and `a_txt` on purpose: a parameter named `sts` or `text` would be replaced inside the designated initializers `.sts = ...` and `.text = ...` too.

The control string. `"Hello, !AZ!! You are visitor number !UL"` is FAO. `!AZ` takes a zero-terminated string, `!UL` an unsigned longword, and `!!` is a literal exclamation mark. A lone `!` followed by a space is *not* a literal: it is an unknown directive, which the engine swallows (with its space) while it sets the status `FAO$K_BADDIR`. A `!` at the very end of the string is the same error, and the scanner stops at the terminating zero instead of reading on. The author of this manual found this out the hard way.

The parameters and the return value. `$PUTMSG_FAO(HELLO$_GREET, "Vasya", 7)` takes the parameters plain: the macro casts each to `fao_prm_t`, a 64-bit unsigned integer, and counts them. It returns the condition value it was given. `HELLO$_NOFILE` has severity `STS$K_ERROR` (2), so the low bit is clear and the program exits with failure. `$LOG_FAO` returns the severity, so `return $LOG_FAO(STS$K_ERROR, ...)` reports and returns in one line.

The two shapes of a record. Without `NDEBUG`, as above:

```
DD-MM-YYYY HH:MM:SS.fff <tid> [MODULE\routine:line] %FAC-S:  text
```

for `$LOG_FAO`, and `... [MODULE\routine:line] %FAC-S-MNEMONIC, text` for `$PUTMSG_FAO`. With `-DNDEBUG` the bracketed part is gone and a `$LOG_FAO` record has one space after the colon, not two:

```
31-10-2026 17:53:14.288 2702736 %HELLO-I: started with 0 arguments
31-10-2026 17:53:14.288 2702736 %HELLO-I-GREET, Hello, Vasya! You are visitor number 7
```

Chapter 4 goes through the parts of a record one by one.

Chapter 3. Condition Values and Messages

3.1 The Layout of a Condition Value

A condition value is an unsigned 32-bit integer in three fields, laid out as on OpenVMS:

```
 31                16 15                            3 2      0
+--------------------+------------------------------+--------+
|  Facility           |  Message number              |Severity|
|  16 bits            |  13 bits                     | 3 bits |
+--------------------+------------------------------+--------+
```

(OpenVMS has a 12-bit facility and four control bits above it. StarLet gives the whole upper half to the facility number, because a Linux program has no use for the control bits.)

Macro	Does
`$DEFFAC(n)`	Puts a facility number in the upper half: `n << 16`
`$DEFMSG(n)`	Puts a message number in bits 3 to 15, masked to 13 bits: `(n << 3) & 0xFFF8`
`$DEFSEV(s)`	Takes the low three bits of a severity
`$FAC(c)`	Extracts the facility: `c >> 16`
`$MSG(c)`	Extracts the message number: `(c >> 3) & 0x1FFF`
`$SEV(c)`	Extracts the severity: `c & 7`
`$STS_ERROR(c)`	True for `STS$K_ERROR` and `STS$K_FATAL`

`$FAC`, `$MSG` and `$SEV` are the exact inverses of the `$DEF...` macros. This was not always so: until version 1.6.3 `$MSG` forgot to mask the facility off. Now `test_status_convention` pins the layout field by field.

A message number runs from 1 to 8191. Zero is better left alone.

3.2 Severities and the Low Bit

Constant	Value	Letter	Low bit	Reads as
`STS$K_WARN`	0	W	0	not successful
`STS$K_SUCCESS`	1	S	1	success
`STS$K_ERROR`	2	E	0	failure
`STS$K_INFO`	3	I	1	success
`STS$K_FATAL`	4	F	0	failure
`STS$K_UNDEF`	5	?	1	(a clamp for out-of-range input: printed as `?`; the log routines return `STS$K_ERROR` for it, never 5)

This is the VMS assignment, and the reason for it is a good one. Severity zero, WARNING, is *even*: a warning is not a success, and a program that tests `1 & status` does not mistake one for a success. It may still decide that a particular warning is harmless - but it has to say so. INFO is odd: a message of information is not a problem.

```
status = __util$base64(src, srclen, dst, dstsz, &dstlen);

if ( !(1 & status) )
    return  $LOG_FAO(STS$K_ERROR, "base64 of !UL octets failed", srclen);
```

Never write `if ( status == STS$K_SUCCESS )`. A routine is allowed to return another odd value as a more specific success, and the day it does, your comparison fails in silence.

3.3 Codes the Library Returns

The routines return plain severities for most results. A few have codes of their own:

Constant	Value	Where	Meaning	Low bit
`FAO$K_NEGTIM`	0x08	time	The result of a time operation would be negative	0
`FAO$K_BUFOVF`	0x0A	FAO, `__util$asctim`	The output was truncated	0
`FAO$K_IVTIME`	0x0C	`__util$bintim`, time	Invalid time string, or an illegal combination	0
`FAO$K_BADDIR`	0x12	FAO	Unknown, malformed or truncated directive	0
`FAO$K_NOPRM`	0x1A	FAO	The control string wants more parameters than were given	0
`FAO$K_FLDOVF`	0x22	FAO	The text of a `!n<...!>` field was longer than 512 octets and was cut	0
`UTIL$S_INVARG`	0x102	queues, `__util$inimsg`	A NULL or unusable argument	0
`UTIL$S_INQUE`	0x202	`$INSQTAIL`, `$INSQHEAD`	The entry is already in another queue	0
`UTIL$S_NOLOCK`	0x302	`$INSQTAIL`, `$INSQHEAD`	The lock of the queue could not be taken	0
`UTIL$S_DUPFAC`	0x400	`__util$inimsg`	A catalogue of this facility is registered already	0

All are even. The FAO and time codes follow the OpenVMS layout (severity in bits 0 to 2, the message number above it, no facility), and from 1.7.0 their numbers are stable. `FAO$K_BADDIR` used to be 6; a program that compared against the number and not against the name must be rebuilt. `UTIL$S_NOLOCK` used to share the value of `UTIL$S_INQUE`; each failure has its own code now. Two of the FAO codes were once odd, so they read as success; in X.01-10 they were renumbered, with a comment explaining why. When a FAO call finds several faults, the **first** one is the status.

3.4 Defining a Facility and Its Messages

A *facility* is a program or a library, with a number. Pick one that other facilities in the same process do not use. The library itself registers nothing, but its default facility name is `"UTILS"`.

A convention that works well, taken from SEARCH:

```
#define MYFAC$K_FACNO   0x0301

#define $MYSTS(msg, sev)    ( $DEFFAC(MYFAC$K_FACNO) | $DEFMSG(msg) | $DEFSEV(sev) )

#define MYFAC$_OPENIN   $MYSTS(3, STS$K_ERROR)
#define MYFAC$_DONE $MYSTS(4, STS$K_INFO)
```

The name of a condition is `FAC$_IDENT`: the facility, a dollar sign, an underscore, the identification of the message. That form matters, because `__util$inimsg` finds the mnemonic after the `$_`.

A message number should be used *once per facility*. The catalogue looks messages up by facility and number, and ignores the severity (see Section 3.6).

3.5 The Catalogue

The catalogue is an array of `EMSG_RECORD` and a descriptor, `EMSG_RECORD_DESC`.

```
typedef struct __emsg_record__ {
    int     sts;        /* the condition value */
    struct { unsigned char textl; char text[254]; };   /* the FAO text, an ASCIC */
    const char  *facname;   /* stamped by __util$inimsg() */
    const char  *mnemonic;  /* optional: "FAC$_NAME", trimmed by __util$inimsg() */
} EMSG_RECORD;

typedef struct __emsg_record_desc__ {
    struct __emsg_record_desc__ *link;  /* used by the library */
    unsigned    msgnr;      /* the number of records */
    unsigned    facno;      /* the facility number */
    struct __emsg_record__ *msgrec; /* the array */
    const char  *facname;   /* the facility name */
} EMSG_RECORD_DESC;
```

The text of a record is limited to 254 octets, and it is an FAO control string - not a `printf` format - so it uses `!AZ`, `!UL` and the rest. Chapter 5 has them all.

You fill the array with designated initializers, as in the first program. The header of a message, by the VMS custom, is its identification: `"NOFILE, no file matches !AZ"`. If a record carries a mnemonic (as `$HELLOREC` made it), the identification is added by the signalling routine, and the text should *not* repeat it. If a record has no mnemonic (SEARCH keeps the identification in the text), the identification is part of the text. Either way is legal. Use one in a given catalogue.

Register the catalogue once, at start-up:

```
unsigned    __util$inimsg   (EMSG_RECORD_DESC *msgdsc);
```

`__util$inimsg` does three things. It **sorts the array in place** with `qsort`, so the lookup can be a binary search: the array must therefore **not be `const`**. It stamps the facility name into each record, and trims the mnemonics. And it links the descriptor into a list.

It returns `STS$K_SUCCESS` if all went well, `UTIL$S_INVARG` if the descriptor is NULL, and `UTIL$S_DUPFAC` if a catalogue with the same facility number is already registered (nothing is changed then). Both failures have the low bit clear. Since 1.7.0 the call is thread-safe: the registrations are serialised, and the list is published only when the descriptor is complete, so a thread that looks a message up meanwhile sees the old list or the new one, never half of it. It is still tidier to register before starting threads.

To find a record without printing it:

```
unsigned    __util$getmsg   (unsigned sts, EMSG_RECORD **outmsg);
```

This returns `STS$K_SUCCESS` and the address of the record, or `STS$K_ERROR` when there is no such facility or message.

3.6 Signalling a Message

```
unsigned    __util$putmsg_fao   (unsigned sts, int prmcnt, ...);
unsigned    __util$putmsgd_fao  (unsigned sts, const char *mod, const char *func,
 ↪  unsigned line,
                int prmcnt, ...);
```

You do not call these directly. You use the macro:

```
$PUTMSG_FAO(sts [, parameter ...])
```

The macro expands to `__util$putmsgd_fao` without `NDEBUG`, and to `__util$putmsg_fao` with it. It casts each parameter to `fao_prm_t` and computes the count from the list (zero to sixteen parameters). The routine

1. finds the record by facility and message number - *ignoring the severity bits*, so one record can be signalled as INFO in one place and as WARNING in another;
2. writes the envelope `%FACILITY-S-MNEMONIC`, using the severity letter of the **caller's** condition value (or `%FACILITY-S-` if the record has no mnemonic);
3. formats the text through `__util$faol`;
4. appends a newline and writes the record, with the usual prefix, in one `write()`;
5. returns the condition value it was given.

So a message of severity ERROR is printed `-E-` and returned as an even number. A caller can write

```
if ( 0 > (l_rc = mkdir(l_path, 0755)) && (errno != EEXIST) )
    return  $PUTMSG_FAO(MYFAC$_MKDIRERR, l_path, errno);
```

and have the report and the status in one expression. A veteran will see in it the shape of `LIB$SIGNAL`, with the one difference that nothing unwinds: the routine just reports and returns.

An unknown condition is not an error to the printing routine. If no record is found, it prints a default line with the numbers decoded and returns the condition anyway:

```
%NONAME-E-NOMSG, Message number: 03010012 [fac: 0x301/769, sev: 0x2/2, msgno: 0x2/2]
```

(Like every other message of the library, the default line begins with the `%`; before 1.7.0 it did not.) Seeing `NOMSG` in a log means a message was signalled whose catalogue was never registered - or whose number is wrong.

The old pair, `$PUTMSG` and `__util$putmsg`, finds the record the same way but treats its text as a `printf` format, and writes no envelope. They are kept for old callers. New code uses `$PUTMSG_FAO`.

A note for the veteran. The real `SYS$PUTMSG` takes a *message vector*, may chain several messages, and calls an action routine. This one takes a condition value and its parameters, and that is all it does. If you miss the message vector, you are in good company.

3.7 Getting the Text Without Printing

A caller who wants the formatted text in a buffer uses `__util$getmsg` for the record and `__util$faol` for the formatting, passing `l_rec->text` as the control string. The result is a counted run of octets **without a terminating zero** (Chapter 5).

Chapter 4. Logging

4.1 The Routines at a Glance

Macro	Routine	Format	Use
`$LOG_FAO(sev, ctl, ...)`	`__util$log_fao`, `__util$logd_fao`	FAO	**Recommended.** One record to the log
`$LOG(sev, fmt, ...)`	`__util$log`, `__util$logd`	`printf`	The older form
`$PUTMSG_FAO(sts, ...)`	`__util$putmsg_fao`, `__util$putmsgd_fao`	FAO, from the catalogue	A message by condition value (Chapter 3)
`$TRACE(fmt, ...)`	`__util$trace`	`printf`	A debug trace; compiled out with `NDEBUG`
`$IFTRACE(cond, fmt, ...)`	`__util$trace`	`printf`	A trace guarded by a condition
`$DUMPHEX(ptr, len)`	`__util$dumphex`	-	Hex and ASCII dump of memory
-	`__util$log2buf`	`printf`	The same record as `$LOG`, into a caller's buffer
-	`__util$out`	`printf`	A raw line, with no prefix

The macros that have a debug variant (`__util$logd`, `__util$putmsgd_fao`) are the ones that add the module, the routine and the line. Which variant is picked is decided by `NDEBUG` **in the translation unit that expands the macro**.

4.2 The Record

The prefix is the same everywhere:

```
31-10-2026 16:22:45.701 1951513 [HELLO\main:34] %HELLO-I:  text
|          |            |       |                |
|          |            |       |                +-- the envelope: %facility-severity
|          |            |       +-- module\routine:line   (without NDEBUG only)
|          |            +-- thread number, six columns
|          +-- time of day, with a fraction of the second
+-- date, DD-MM-YYYY, in the local time zone
```

Producer	After the prefix	Notes
`$LOG_FAO`, `$LOG` (debug)	`[MOD\func:line] %FAC-S: text`	Two spaces after the colon
`$LOG_FAO`, `$LOG` (NDEBUG)	`%FAC-S: text`	One space
`$PUTMSG_FAO`	`[MOD\func:line] %FAC-S-IDENT, text`	Without NDEBUG; no bracket with it
`__util$log2buf`	`%FAC-S:text`	No space, no newline; into your buffer
`$TRACE`	`[MOD\func:line]`, spaces to column 72, text	Nothing at all with NDEBUG
`$DUMPHEX`	`[func:line] Dump of N octets follows:`	Then lines `\t+0000:  48 65 ...`

Things worth knowing:

- **The fraction of the second is milliseconds**, 000 to 999, on every platform. (Before 1.7.0 it was the nanoseconds shifted right by 20 bits, which ran from 000 to about 953 and, on Windows, printed six digits.) The clock is the coarse one, `CLOCK_REALTIME_COARSE`: cheap, and good to a few milliseconds, so the last digit is not a measurement.
- **The date and time are local time**, from `localtime_r`. (The FAO directives `!%D` and `!%T` print UTC. Chapter 5 and Chapter 10 explain.)
- **The thread number** is the kernel's, obtained once per thread. After a `fork()` the child inherits the cached number; if the child cares, it should re-exec or reset it.
- **A record has an upper limit of 2047 octets** (and the newline). Longer ones are truncated inside the buffer; `test_log_overflow` checks that the terminating newline always lands inside it.
- **One `write()` per record.** The buffer is built, then written whole. Two threads do not interleave within a line. (`$DUMPHEX` writes a line at a time.)

4.3 $LOG_FAO

```
unsigned __util$log_fao (const char *fac, unsigned sev, const char *ctl, int prmcnt, ...);
```

You write:

```
$LOG_FAO(STS$K_WARN, "queue !AZ holds !UL entr!%S", l_name, l_count);
```

and the macro supplies the facility (`__FAC__`), the module, routine and line (without `NDEBUG`), the casts and the count. Up to sixteen parameters. The severity is one of `STS$K_*`; the letter printed is `W`, `S`, `E`, `I` or `F`. The routine returns the severity, so

```
return  $LOG_FAO(STS$K_ERROR, "no room for !UL octets", l_size);
```

reports and returns an even number in one line. And since the severity of INFO is 3,

```
return  $LOG_FAO(STS$K_INFO, "all done");
```

returns an odd one: a success. This is a neat idiom, and the library uses it itself.

The routine returns the severity it logged, so the value is always the right one to return.

If the FAO formatter finds a fault in the control string - a bad directive, too few parameters, a field or the output cut - the record is still written, with a short marker at its end, **`[FAO BADDIR]`**, `[FAO NOPRM]`, `[FAO FLDOVF]` or `[FAO BUFOVF]` (another status shows as `[FAO 0x...]`), so a typo is seen at once. The return value does not change. `$PUTMSG_FAO` does the same. (Before 1.7.1 the status was dropped.) A severity out of the range 0..4 is printed as `?`, and the routine returns **`STS$K_ERROR`** (low bit clear): an undefined severity is a caller's error and must not read as success (before 1.7.1 the return was 5, which does). The syslog bit (Section 4.6) is taken off *before* that test, so a call flagged for syslog returns the plain severity as well; before 1.7.0 it returned 5, which reads as success, for every flagged call. The same holds for `$LOG`, in a debug unit and in an `NDEBUG` one.

`$LOG` is the same with a `printf` format string and `printf` arguments. Use it only where the parameters of FAO would not help.

4.4 Where the Log Goes

By default, to the standard output. Two routines change it.

```
int __util$deflog    (const char *logfile, const char *sloghost);
int __util$setlogfd  (int fd);
```

__util$deflog opens `logfile` for appending (creating it, mode `rw-r--r--`, close-on-exec), makes it the log, and *also* `dup2`s it over the standard output and the standard error, so a stray `printf` lands in the same file. It then logs `Log file '...' has been opened`. A NULL or empty name leaves things as they are and succeeds. If it is called again, it closes the file it opened before. It is the right call for a daemon.

The second argument, `sloghost`, is an IPv4 address text for a UDP syslog collector, port 514. A text that is not an IPv4 address is reported and ignored. **__util$deflog only remembers that address.** No log routine sends a record to it, whatever the severity and the syslog bit: `STS$M_SYSLOG` goes to the local `syslog(3)` (Section 4.6). The collector is the destination of `__util$syslog`, which you call yourself. Pass NULL if you have no collector.

__util$setlogfd points the log at a descriptor that is already open, and touches nothing else: no file is opened, no `dup2`, nothing seized. The classic use is a filter whose standard output carries data:

```
#include <unistd.h>

__util$setlogfd(STDERR_FILENO);
```

From that moment `$LOG`, `$PUTMSG_FAO` and `$TRACE` go to the standard error and the data stays clean. It returns `STS$K_ERROR` for a negative descriptor. The descriptor is yours; the library never closes it, and never rotates it.

4.5 Rotation

A daemon that logs for months needs its log kept in check. Two routines do it. Call either one now and then - once in a few seconds is plenty.

```
int __util$rotatelogfile (size_t limit, unsigned keep);
int __util$rewindlogfile (size_t limit);
```

`__util$rotatelogfile(limit, keep)` is the good one. When the file opened by `__util$deflog` has grown to `limit` octets, it renames `file.<keep-1>` to `file.<keep>`, and so on down to `file` to `file.1` (the oldest is dropped), then opens a fresh `file` on the very same descriptor, and on the standard output and error with it. Nothing is copied: a rename is a metadata change, so a slow disk does not stall the program, and nothing is lost. A record that another thread writes meanwhile lands at the end of the old file or at the start of the new one, never torn. `keep` runs from 0 to 99; 0 truncates in place. A limit of zero, a log that goes to the standard output, or a descriptor given by `__util$setlogfd` mean “nothing to do”. Only one thread rotates at a time; the others see it under way and go on.

`__util$rewindlogfile(limit)` is the crude one: past the limit it rewinds and truncates the file. The old records are lost. It checks the size once, at the position of the descriptor. Keep it for tools where a log is a scratch pad.

```
__util$deflog("/var/log/myd.log", NULL);

for ( ;; )
    {
    /* ... the work of the daemon ... */
    __util$rotatelogfile(10 * 1024 * 1024, 5);  /* 10 MB, keep 5 */
    }
```

4.6 Syslog and Logcat

OR STS$M_SYSLOG (16) into the severity, and the record is *also* sent to the system syslog(3). The priority is LOG_INFO for a severity with the low bit set (SUCCESS, INFO) and LOG_ERR for the others. This works on non-Windows systems, for $LOG and $LOG_FAO, in a debug unit and in an NDEBUG one alike. (Before 1.7.0 only the NDEBUG form did it, and the priority always came out as LOG_INFO, with LOG_PID mixed in by mistake.) __util$log2buf writes to a buffer and never to syslog.

The UDP collector named in __util$deflog is not fed by STS$M_SYSLOG; to send a datagram to it, call the routine yourself:

```
unsigned __util$syslog (int fac, int sev, const char *tag, const char *msg, int msglen);
```

It sends <fac+sev> tag: msg to port 514. With no collector set it does nothing and returns STS$K_SUCCESS; it writes no diagnostics of its own. It returns STS$K_ERROR if the socket cannot be made or the send fails. The socket is made once, on the first use, under a guard that threads may race through.

On Android, define ANDROID_LOGCAT when compiling the library to have each record written to logcat as well (Chapter 12).

4.7 Tracing and Dumping

```
$TRACE("opening %s, flags=%#x", "/etc/hosts", 2);
```

`$TRACE` takes a `printf` format. `$TRACE` and `$IFTRACE` compile to nothing with `NDEBUG`, so tracing costs nothing in a release. With `NDEBUG` the arguments are not even evaluated; `$IFTRACE` keeps its condition in a `sizeof`, so a variable used only in a trace does not draw an “unused” warning.

`$DUMPHEX(ptr, len)` writes a dump of up to 65535 octets, sixteen to a line:

```
31-10-2026 16:24:27.227 1964971 [main:8] Dump of 18 octets follows:
    +0000:  48 65 6c 6c 6f 2c 20 53 74 61 72 4c 65 74 21 0a   Hello, StarLet!.
    +0010:  01 02                                             ..
```

`__util$faohex(src, srclen, out, outsz, &outlen)` formats one such line (at most 16 octets of the source; the hex and the text, 67 octets of room needed) into your own buffer, if you want to dump to somewhere other than the log. It returns the length written, or 0 if the buffer is too small or an argument is NULL; `outlen` may be NULL since 1.7.0.

`$TRACE` is a debug tool. For something that must always be reported, use `$LOG_FAO`.

4.8 Records into a Buffer

```
unsigned __util$log2buf (void *out, int outsz, int *outlen, const char *fac, unsigned sev,
            const char *fmt, ...);
```

Formats a record exactly as `$LOG` would, but into `out` - no newline, a terminating zero, always inside `outsz`, and the length in `*outlen`. It is for the case where the text of the record is needed as a string. The string has the form `date time tid %FAC-S:text`. Like `$LOG` it returns the severity logged.

`__util$out(fmt, ...)` writes a raw line to the log descriptor: no prefix, newline added. The help generator of the command parser uses it, so help goes where the log goes.

Chapter 5. FAO, the Formatter

5.1 Why FAO

`SYS$FAO` is one of the best small ideas of OpenVMS. The *control string* describes the output; the *parameters* are the data; nothing depends on a compiler checking a format against types, because the directive says exactly how big each parameter is. The same directives work in a log record, in a message, on a screen and in a protocol field. StarLet's engine, `__util$fao` and `__util$faol`, is a portable rewrite of it in plain C. It does not use `<stdio.h>`, nor `<string.h>`, nor `<ctype.h>`.

```
int __util$fao  (const char *ctlstr, unsigned short *outlen, fao_desc_t *outbuf, ...);
int __util$faol (const char *ctlstr, unsigned short *outlen, fao_desc_t *outbuf,
         const fao_prm_t *prmlst, int prmcnt);
```

`outbuf` is a descriptor over your buffer - a small struct `fao_desc_t`, in the family of the OpenVMS `dsc$descriptor_s` (the length word, two unused bytes of type and class, the address). The initializer is `$FAO_DESC(buf)` for a char array. `*outlen` receives the length of the result. **The result is not terminated with a zero**, as on OpenVMS; add one yourself if you need it.

The return value is a condition value: `STS$K_SUCCESS`, or the **first** fault found - the text is produced all the same. `FAO$K_BUFOVF` means the output was truncated to the buffer; `FAO$K_BADDIR` that a directive was unknown, malformed or cut short by the end of the string; `FAO$K_NOPRM` that the control string wanted more parameters than it was given; `FAO$K_FLDOVF` that the text of a `!n<...!>` field was longer than 512 octets and was cut. `STS$K_ERROR` means a NULL argument. The engine never writes past the end of the buffer, and never reads past the terminating zero of the control string. `outlen` may be NULL.

`__util$faol` takes the parameters as an array and a count. A directive that asks for more parameters than the array has gets zero, and the status is `FAO$K_NOPRM`. `__util$fao` takes them as a variable argument list, and cannot know how many you passed: if the control string asks for more than you gave, the behaviour is undefined, and `FAO$K_NOPRM` is raised only when more than 64 parameters are wanted. This is the reason the macros `$LOG_FAO` and `$PUTMSG_FAO` exist, and the reason they use `faol`.

5.2 Parameters

Every parameter is a `fao_prm_t`, an unsigned 64-bit integer **passed by value**. A pointer, a string address, a time, a 64-bit number - all are passed in it. With `__util$fao` you cast each one yourself:

```
char        l_buf [128];
fao_desc_t  l_dsc = $FAO_DESC(l_buf);
unsigned short  l_len;

__util$fao("!AZ has !UL file!%S", &l_len, &l_dsc, (fao_prm_t) "BACKUP", (fao_prm_t) 3);
```

With the macros you write them plain. A quadword is passed by value, and has been since X.01-09.

A quirk for the veteran: on OpenVMS every parameter is a longword and an address. Here a quadword directive reads the whole 64 bits of the parameter. Cast a signed value through its own width first: `(fao_prm_t)(int32_t) -42`.

5.3 The Directives

A directive begins with `!`. Between the `!` and the letters you may put a number - a *repeat count* or a *field width* - or `#`, which takes the number from the parameter list.

Strings

Directive	Parameter	Output
`!AZ`	Address of a zero-terminated string	The string. A NULL address prints nothing
`!AS`	Address of an `fao_desc_t`	The text the descriptor describes
`!AC`	Address of a counted string (`ASC`)	The text after the length octet
`!AD`	Two: length, then address	That many octets
`!AF`	Two: length, then address	As `!AD`, but a non-printable octet shows as `.`

A field width, as in `!12AZ`, pads with blanks on the right, or cuts on the right: `!12AZ` of `zero-term` gives `zero-term`, and `!4AZ` gives `zero`.

Numbers. The second letter is the size: **B** byte, **W** word, **L** longword, **Q** quadword.

Directive	Output
`!UB !UW !UL !UQ`	Unsigned decimal
`!SB !SW !SL !SQ`	Signed decimal (the value is sign-extended from its size)
`!XB !XW !XL !XQ`	Hexadecimal, upper case, zero-filled to 2, 4, 8, 16 digits
`!OB !OW !OL !OQ`	Octal, zero-filled to 3, 6, 11, 22 digits
`!ZB !ZW !ZL !ZQ`	Unsigned decimal, zero-filled to the field width (natural width if none)

With a width `n`, a decimal number is right-justified in the field, blank-filled; `!6UL` of 42 is `42`. If it does not fit, the field is filled with asterisks: `!2UL` of 12345 is `**`. A hex or octal number that does not fit is cut on the *left*: `!4XL` of `0x12345678` is `5678`. That is the OpenVMS rule, and it is easy to be surprised by it.

Control

Directive	Output
`!/`	Carriage return and line feed (CR LF, as in OpenVMS)
`!_`	Horizontal tab
`!^`	Form feed
`!!`	A literal `!`
`!%S`	The letter `s`, if the last number converted was not 1
`!n*c`	The character `c`, repeated `n` times (once if no `n`)
`!n(...!)`	The enclosed directives, repeated `n` times
`!n<...!>`	The enclosed directives formatted into a field of `n` octets, blank-filled or cut on the right. The text inside may be up to 512 octets; more is cut and sets `FAO$K_FLDOVF`

Directive	Output
`!n-`	Step back over `n` parameters (one if no `n`): the next directive reads the same ones again
`!n+`	Skip `n` parameters

`!/` is CR LF. A Unix stream expects a bare LF, so programs on Linux that print lines write `\n` themselves. The log routines add a `\n` for you.

Groups nest to a depth of four. An unterminated or too deep group sets `FAO$K_BADDIR`, and so does a directive that the string ends in the middle of (`!`, `!A`, `!%`, `!3*`).

Time and date. The parameter is a time by value: seconds of the UNIX epoch. The sign decides:

Parameter	`!%D`	`!%T`
Positive	`dd-MMM-yyyy hh:mm:ss.cc`, the day blank-padded to two, the month in capitals	`hh:mm:ss.cc`
Zero	The current time. The epoch itself cannot be shown; one second after it can	The current time
Negative	A delta of that many seconds: `dddd hh:mm:ss.cc`	`hh:mm:ss.cc`

So a time before 1970 cannot be written as an absolute time: a negative number is a delta, always. A year above 9999 is printed in full (`1-JAN-10000 00:00:00.00`); before 1.7.0 it read past its table of digits. The hundredths are always `.00`. The time is UTC plus a constant, `FAO$K_TZOFFSET`, fixed when `starlet.c` is compiled (zero by default). It is not local time, which is a difference from the log record prefix.

Addresses. The parameter is the address of the raw octets. A NULL address prints `(null)`.

Directive	Reads	Output
`!%M`	6 octets	A MAC address, `00:1b:44:11:3a:b7`
`!%I`	4 octets	An IPv4 address, `192.168.42.2`
`!%6`	16 octets	An IPv6 address in the RFC 5952 canonical form, `2001:db8::1`; an IPv4-mapped one as `::ffff:192.168.1.10`
`!%U`	16 octets	A UUID, `12345678-9abc-def0-1122-334455667788`

The text is in lower case. The `!%6` form is byte-identical to `inet_ntop()`, and `test_fao_addr` holds it to that: the longest run of two or more zero groups becomes `::`, the leftmost on a tie, and a single zero group is never compressed.

5.4 A Tour of Real Output

Every line below was produced by the code (a `T(ctl, ...)` helper calling `__util$fao`):

```
!UL          42       -> [42]
!XL          255      -> [000000FF]
!5ZL         7        -> [00007]
!6UL|        42       -> [    42|]
!2UL         12345    -> [**]
!4XL         0x12345678 -> [5678]
!AC          "VMS-8"  -> [VMS-8]
!AD          2, "zero-term" -> [ze]
!12AZ|                -> [zero-term   |]
!UL file!%S  1        -> [1 file]
!UL file!%S  3        -> [3 files]
!8<abc!>|             -> [abc     |]
!UL !-!XL    255      -> [255 000000FF]
!#UL|        6, 9     -> [     9|]
!%M                   -> [00:1b:44:11:3a:b7]
!%D          1793462400 -> [31-OCT-2026 16:00:00.00]
!%D          -93784     -> [1 02:03:04.00]       a delta: one day, 2 h 3 min 4 s
!%D          253402300800 -> [ 1-JAN-10000 00:00:00.00]
!UL !UL      1 (one parameter given) -> [1 0]    status FAO$K_NOPRM (0x1A)
!QL                   -> [L]       status FAO$K_BADDIR (0x12)
Ab!A             -> [Ab]    status FAO$K_BADDIR (0x12): the string ends inside the directive
(a 16-octet text into an 8-octet buffer) -> [01234567]  status FAO$K_BUFOVF (0x0A)
```

Look at the last four. A bad directive does not stop the engine: it sets the status and goes on. A parameter that was not given reads as zero. A buffer that is too small is filled to the brim, and the status says so. Only the first fault is reported; the later ones are not lost, just not counted.

5.5 FAO in Messages and Logs

Everything in Chapters 3 and 4 uses this engine. Its parameters are passed by the macro. For example

```
$LOG_FAO(STS$K_INFO, "client !%I:!UL connected from !%M", l_ip4, (unsigned) l_port,
↪  l_mac);
```

where `l_ip4` is an array `unsigned char[4]` and `l_mac` an array `unsigned char[6]`. The macro casts the arrays to `fao_prm_t`, which for an array is the address of its first octet.

Two limits of the macro form: at most sixteen parameters, and a formatted body of at most 2047 octets.

5.6 Gotchas

- **A lone `!` is a directive.** To print an exclamation mark, write `!!`. A `!` that ends the string is `FAO$K_BADDIR`, and nothing beyond the terminating zero is read.
- **Quadwords by value.** `!UQ`, `!XQ` and `!SQ` read the parameter as it is.
- **The count is yours with `__util$fao`.** The macros count for you; the varargs routine cannot.
- **`!AZ` of a NULL pointer prints nothing**, and `!AS`/`!AC` of NULL likewise; the address directives print `(null)`.
- **No trailing zero.** The output is a counted run of octets.
- **The engine reports its own errors silently.** Inside `starlet.c` the `$LOG` macro is empty, so a bad directive is reported only through the status (`FAO$K_BADDIR`, `FAO$K_NOPRM`, …). `$PUTMSG_FAO` and `$LOG_FAO` do not check that status, so a typo in a control string produces damaged output and no complaint. Test your messages once, by looking at them.
- **A time of zero is “now”.** A report of an event that happened at the epoch will show the clock of the moment instead. A negative time is a delta, never a date before 1970.

Chapter 6. Strings and Data

6.1 The Counted String

On OpenVMS a string is often carried as an ASCIC: one octet of length, then the text. It is cheap, it needs no scan to find its end, and it can hold a zero octet. StarLet's `ASC` is that string:

```
typedef struct _asc {
    unsigned char   len;        /* 0..255 */
    char        sts [255];  /* the text */
} ASC;
```

Macro	Does
`$ASCDECL(a, "text")`	Declares and initializes an `ASC` named `a` from a literal
`$ASCINI("text")`	The initializer list `len, "text"`; use it inside braces in a table
`$ASCLEN(p)` / `$ASCPTR(p)`	The length, and the address of the text, of an `ASC *`
`$ASC(p)`	Expands to `len, text`: the two arguments `"%.*s"` wants
`$ASCNIL`	`0, 0` - an empty entry

```
$ASCDECL(l_name, "counted");
char    l_z [64];

__util$asc2str(&l_name, l_z);
$LOG_FAO(STS$K_INFO, "name is !AC, length !UL", &l_name, (unsigned) $ASCLEN(&l_name));
```

The `!AC` directive of FAO takes the address of an `ASC` directly.

The routines:

Routine	Does
`__util$str2asc(const char *src, void *dst)`	Copies a C string into an `ASC` and terminates it; a text over 254 octets is cut there, so that the zero keeps its place inside `sts[]`; returns the length stored
`__util$asc2str(ASC *src, char *dst)`	Copies an `ASC` to a C string and terminates it; returns the length
`__util$cmpasc(a, b)`	Compares two `ASC`: the difference of the lengths first, then the octets
`__util$cmpasc_blind(a, b)`	The same, ignoring case
`__util$cpyasc(src, dst)`	Copies one `ASC` to another

Routine	Does
`__util$strelem(src, sep, idx, dst)`	The `idx`-th piece (from 0) of `src` split at `sep`, into an `ASC`; returns its length. For a negative index, or a string with fewer pieces, `dst` is left empty and the result is 0

Remember what “compare” means here: it is a three-way compare in which a shorter string always sorts before a longer one. It is the right test for *equal*, and a poor one for dictionary order.

6.2 Moving and Searching

```
static inline int __util$movc5 (unsigned short *srclen, char *source, unsigned char *fill,
                unsigned short *dstlen, char *dest);
```

The MOVC5 instruction of the VAX as a function: move the source to the destination; if the destination is longer, pad it with `*fill`; if shorter, the extra source octets are not moved; overlap is safe. The lengths are passed by address, as the VAX did it.

`char *__util$strstr(char *s1, size_t s1len, char *s2, size_t s2len)` finds `s2` in `s1` with both lengths given, and returns the address of the match or NULL. It never looks past `s1len`.

`__util$strcat_rx(dst, dstsz, &dstlen, src1, len1, src2, len2, ..., NULL)` joins a list of pieces, each given as an address and a length, into one buffer, and terminates the list with a NULL address. It returns `STS$K_SUCCESS` when all went in, and `STS$K_ERROR` if a piece does not fit (the pieces before it stay, and the result is terminated) or an argument is bad. One octet is kept for the terminating zero, so the pieces may fill the buffer up to `dstsz - 1` octets exactly, and nothing is ever written at `dst[dstsz]`. A piece with a zero length is skipped.

`__util$fill(dst, dstsz, pattern, patternsz)` fills a buffer by repeating a pattern; the last copy is cut to fit. It returns `STS$K_ERROR` for a NULL address, a negative size, or a pattern of no length (which could never fill anything). `__util$strxor(key, keysz, src, srcsz, dst, dstsz, ctx)` XORs data with a repeating key; with `ctx` non-NULL it remembers the position in the key between calls. It is a mask, not a cipher.

6.3 Cleaning Up Text

All three work in place and return the new length.

```
char    l_line [] = "  key = \"a ! b\" ! a comment  \n";
int l_len;

l_len = __util$uncomment(l_line, (int) strlen(l_line), '!');    /* cut at the first '!'
 ↪  outside quotes */
l_len = __util$trim(l_line, l_len);             /* drop blanks at both ends */
```

Routine	Does
`__util$trim(s, len)`	Removes leading and trailing white space (`isspace`)
`__util$collapse(s, len)`	Removes *all* white space. The result is terminated when something was removed; the buffer of `len` octets is never overrun
`__util$uncomment(s, len, marker)`	Cuts the string at the first `marker` octet that is not inside quotes

A part in double or single quotes is skipped as it stands. The other kind of quote inside it is an ordinary octet - there is no escape - and a quote left open runs to the end of the string. In the example the `!` inside the quotes survives and the one after them starts the comment; the result is `key = "a ! b"`. The routine returns 0 if nothing but a comment was there.

They are what a configuration reader needs, and `__util$readconfig` is built from them.

6.4 Wildcards

```
int __util$pattern_match (char *str, char *pattern);
```

Returns 1 for a match and 0 for none (also for a NULL argument) - not a condition value. `*` matches any run of octets (including none), `%` matches exactly one, and `?` is accepted as an alias of `%`. The comparison is case-sensitive, the whole string must match, and there is no escape: a literal `?` in a pattern is a wildcard. The wildcards belong to the *pattern* only. A `*` in the subject string is an ordinary octet; `"*ba"` matches the pattern `"*a"`, as it does `"?ba"` (before 1.7.0 it did not).

```
__util$pattern_match("report.txt", "*.txt");    /* 1 */
__util$pattern_match("a1c", "a%c");     /* 1 */
__util$pattern_match("abc", "a?d");     /* 0 */
```

The matcher is iterative - no recursion, and no stack to exhaust - so its running time is bounded by the product of the two lengths. A hostile pattern of many stars, which once took 8.6 seconds, takes a few hundred nanoseconds. Patterns from a configuration file or from a user are safe to match.

6.5 Base64

```
int __util$base64   (void *src, int srclen, void *dst, int dstsz, int *dstlen);
int __util$unbase64 (void *src, int srclen, void *dst, int dstsz, int *dstlen);
```

RFC 4648. The same bounded calling sequence as the rest of the family: source and length, destination and its size, and the length produced. `*dstlen` is set even when the buffer is too small, so a caller can learn what size it needs. The encoder pads with `=` and adds a terminating zero when there is room. The decoder accepts padded and unpadded input, and refuses everything else: an octet outside the alphabet, a `=` in the middle, a dangling sextet. It returns `STS$K_SUCCESS` or `STS$K_ERROR`.

```
char    l_out [32];
int l_len;

if ( 1 & __util$base64("Man", 3, l_out, sizeof(l_out), &l_len) )
    $LOG_FAO(STS$K_INFO, "encoded: !AD", l_len, l_out); /* TWFu */
```

6.6 Hexadecimal and UUID Text

Routine	Does
`__util$bin2hex(src, dst, n)`	`n` octets to `2n` lower-case hex digits **and a terminating zero** (room for `2n + 1`); returns `2n`. Macro `$BIN2HEX(s, d, l)` casts
`__util$hex2bin(src, dst, n)`	`n` hex digits (either case) to `n/2` octets; returns that count. An odd `n` or a character that is not a hex digit gives **-1** and nothing is written; `n` 0 gives 0
`__util$uuid2str(uuid, buf, bufsz)`	16 octets to the 36-character text, lower case; returns the length
`__util$faohex(src, n, out, outsz, &outlen)`	One 16-octet dump line (Section 4.7); `outlen` may be NULL

6.7 CRC-32

```
unsigned int __util$crc32c (unsigned int crc, const void *buf, size_t buflen);
```

The name says Castagnoli, but the routine computes the **IEEE 802.3 CRC-32** (the zlib one, reflected polynomial `0xEDB88320`). The check value of `"123456789"` is `0xCBF43926`. This is by design: the name was in use by programs before anyone noticed, and the header now says so in plain words. Start with `crc = 0`, and to continue a running checksum pass the previous result back as `crc`:

```
unsigned l_crc = 0;

l_crc = __util$crc32c(l_crc, l_block1, l_len1);
l_crc = __util$crc32c(l_crc, l_block2, l_len2);
```

It returns the checksum itself - this routine does not use the condition-value convention. On its first call it chooses its engine: PCLMULQDQ folding on x86-64 when the processor has it, the CRC32 instructions on ARMv8 when `AT_HWCAP` says so, the slice-by-8 tables otherwise. All give the same bits. Setting the environment variable `STARLET_CRC32=SOFTWARE` forces the tables, which the test uses to check them too.

6.8 Item Lists

An item list is a table of (code, length, address) entries that tells a service what to fetch or set. OpenVMS system services take them everywhere. StarLet has the types:

```
typedef struct __ile2 { unsigned short code, len; char *ptr; } ILE2;
typedef struct __ile3 { unsigned short code, len; char *ptr; unsigned short *retlen; }
 ↪  ILE3;
```

and `$ILE2INI(code, len, ptr)` as an initializer, `$ILENULL` as the terminator. A word of caution for the veteran: the ILE3 of OpenVMS puts the *length* word first and the *code* second; this one puts the code first, and its pointers are the width of a Linux pointer. It is a shape, not a binary layout, and a list built here must not be handed to a system service of OpenVMS as it stands. The header says so too.

`$ILE3INI(item, code, len, ptr, retlen)` fills one ILE3 *object* (not a pointer to it) and is a statement, to be followed by a semicolon. It compiles with ordinary arguments since 1.7.0; before that its parameter names clashed with the names of the members.

```
ILE3        l_items [3];
char        l_buf [32];
int     l_num = 0;
unsigned short  l_len = 0;

$ILE3INI(l_items[0], 1, sizeof(l_buf), l_buf, &l_len);
$ILE3INI(l_items[1], 2, sizeof(l_num), &l_num, NULL);
l_items[2] = (ILE3) { 0 };
```

6.9 Small Things

- `$ARRSZ(a)` - the number of elements of an array.
- `$MIN(x, y)`, `$MAX(x, y)`, `$RANGE(x, lo, hi)`, `$ISINRANGE(x, lo, hi)` - on `int`s. `$ISINRANGE2` is the same test; both include the borders, in whatever order they are given (`__util$isinrange2` left the borders out until 1.7.0, against its own comment).
- `__util$iszero(buf, n)` - `STS$K_SUCCESS` if all octets are zero, `STS$K_WARN` otherwise. A NULL buffer or a length of zero has nothing to show and is "all zero".
- `__util$uint32_2_dec(value, dst, dstsz, &len)` - decimal text of an unsigned value, bounded.
- `__util$bzero(buf, n)` - clears every one of the `n` octets; `STS$K_ERROR` for a NULL buffer with a non-zero length. (Until 1.7.0 it left the last one to three octets alone.)
- `__util$mask2sym` and `__util$val2sym` count the separator and the terminating zero when they check the room, so a result that does not fit is cut short but never runs over.

Chapter 7. Keywords and Parameters

7.1 Looking up a Keyword

A program often has a small set of words - the verbs of a console, the names of modes - and needs the value behind a word the user typed. `__util$lookup_key` does it against a table of `KWDENT`:

```
typedef struct _kwdent {
    ASC kwd;            /* the keyword, an ASCIC */
    union { int val; void *ptr; };  /* what it stands for */
} KWDENT;

/* an inline routine of utility_routines.h: */
int __util$lookup_key (const char *src, int srclen, KWDENT *tbl, int tblsz, KWDENT **kwd,
 ↪  int flags);
```

Flag	Meaning
`UTIL$K_LOOKUP_NCASE`	Compare without regard to case
`UTIL$K_LOOKUP_ABBREV`	The string may be a leading part of a keyword, as DCL takes them

With no flags the string must equal a keyword in length and in every octet. This is the behaviour since 1.6.11, when the ABBREV mode was corrected (it had run past the end of the table):

- **A keyword spelled out in full wins**, whatever else the string is an abbreviation of. `SET` is `SET` even though `SETUP` also begins with `SET`.
- **An abbreviation selects a keyword if it selects only one.** `SETU` is `SETUP`.
- **Synonyms are not ambiguous.** Two entries with the *same value* count as one. A table may list `BOOT` and `B` with the same value, and `bo` still finds BOOT.
- **An ambiguous abbreviation is `STS$K_WARN`.** `*kwd` is set to the *first* keyword it matches, so a program that wants to be lenient can use it; one that wants to be strict refuses.
- **Nothing found, or an empty string, is `STS$K_ERROR`**, and `*kwd` is left untouched.
- The entry returned is always inside the table.

```
enum { K_BOOT = 1, K_SHOW, K_SET, K_SETUP };

static KWDENT   s_kwd [] = {
    { {$ASCINI("BOOT")},  {.val = K_BOOT} },
    { {$ASCINI("SHOW")},  {.val = K_SHOW} },
    { {$ASCINI("SET")},   {.val = K_SET} },
    { {$ASCINI("SETUP")}, {.val = K_SETUP} },
    };
```

```
KWDENT  *l_kwd = NULL;
int l_sts;

l_sts = __util$lookup_key("sho", 3, s_kwd, $ARRSZ(s_kwd), &l_kwd,
              UTIL$K_LOOKUP_NCASE | UTIL$K_LOOKUP_ABBREV);

if ( l_sts == STS$K_WARN )
    return  $LOG_FAO(STS$K_ERROR, "ambiguous keyword !AD", 3, "sho");
if ( !(1 & l_sts) )
    return  $LOG_FAO(STS$K_ERROR, "unknown keyword !AD", 3, "sho");
```

Here `sho` gives `K_SHOW` with `STS$K_SUCCESS`, and `se` gives `STS$K_WARN` with `*kwd` pointing at `SET`. Note the test: a WARN is *not* a success (the low bit is clear), so `1 & l_sts` refuses it.

7.2 Values to Words

For a log or a display the other direction is needed: the word for a value. A `SYM_REC` table pairs a value with its name, built by `$SYM_REC_INI(NAME)`, which stringizes the name:

```
#define LNK$M_UP    1
#define LNK$M_ERR   2

static const SYM_REC    s_flags [] = {
    $SYM_REC_INI(LNK$M_UP), $SYM_REC_INI(LNK$M_ERR), $SYM_REC_EOL
    };

char    l_text [64];

__util$mask2sym(3, s_flags, l_text, sizeof(l_text));    /* "LNK$M_UP|LNK$M_ERR" */
__util$val2sym(2, s_flags, l_text, sizeof(l_text)); /* "LNK$M_ERR" */
```

`__util$mask2sym` joins the names of all the bits set with `|`; `__util$val2sym` finds the name of one exact value. Both return the length of the text and terminate it. If the buffer is too small, the output stops short rather than overflow. Both are good for the `!AD` of a log.

7.3 Options of the Command Line and of a File

The same table describes the options of a program, for the command line and for a configuration file. One `OPTS` entry names an option, says where its value is stored and what type it is:

```
typedef struct _opts {
    ASC name;       /* the option's name */
    void    *ptr;       /* where the value goes */
    size_t  sz;     /* the size of that place */
    int type;       /* OPTS$K_* */
} OPTS;
```

Type	The variable is	The text parsed
`OPTS$K_INT`	An integer of exactly `sz` octets: 1, 2, 4 or 8 (0 means `int`)	A number, decimal, `0x` hex or `0` octal, with nothing after it. Size 1, 2, 8: **unsigned**, a minus sign is refused, 0 to 2^n-1. Size 4 and 0: **signed**, a minus sign is taken, the int32 range
`OPTS$K_STR`	An `ASC`	Any text, cut at the smaller of `sz` and 254 octets
`OPTS$K_PWD`	An `ASC`	As `STR`; `__util$showparams` hides it
`OPTS$K_OPT`	`int`	A switch: the presence sets it to 1
`OPTS$K_PORT`	`unsigned short`	A port number, 0 to 65535, stored in **host** byte order; use `htons()` to put it on the wire
`OPTS$K_IP4`	`struct in_addr`	A dotted IPv4 address, stored as `inet_aton` makes it
`OPTS$K_CONF`	An `ASC`, or NULL	The name of a configuration file, read at once; not allowed inside a configuration file

An `OPTS$K_INT` of any other size is an error, and the variable is left alone. (Before 1.7.0 it was written as an `int`, which overran a `short` and was too small for a `long`.) From 1.7.1 the sign rule holds: an option of 1, 2 or 8 octets is unsigned (`-1` is a bad value, not `0xFF`), and one of 4 octets or `int` is signed, from -2147483648 to 2147483647 (`0xFFFFFFFF` is a bad value; put a 32-bit mask into an 8-octet variable). `__util$showparams` prints them the same way.

The table ends with `OPTS_NULL`. A complete program:

```
#define __MODULE__  "OPTT"
#include    <stdio.h>
#include    <arpa/inet.h>
#include    "utility_routines.h"
#include    "starlet.h"

static  ASC     g_conf, g_name, g_pwd;
static  int     g_trace, g_count;
static  unsigned short  g_port = 4021;
static  struct in_addr  g_addr;
static  unsigned long long g_big;
```

```
static  const OPTS  s_opts [] = {
    { {$ASCINI("config")},   &g_conf,  sizeof(g_conf),  OPTS$K_CONF },
    { {$ASCINI("name")},     &g_name,  sizeof(g_name),  OPTS$K_STR },
    { {$ASCINI("password")}, &g_pwd,   sizeof(g_pwd),   OPTS$K_PWD },
    { {$ASCINI("trace")},    &g_trace, sizeof(g_trace), OPTS$K_OPT },
    { {$ASCINI("count")},    &g_count, sizeof(g_count), OPTS$K_INT },
    { {$ASCINI("big")},      &g_big,   sizeof(g_big),   OPTS$K_INT },
    { {$ASCINI("port")},     &g_port,  sizeof(g_port),  OPTS$K_PORT },
    { {$ASCINI("server")},   &g_addr,  sizeof(g_addr),  OPTS$K_IP4 },
    OPTS_NULL
    };

int main    (int argc, char *argv[])
{
    if ( !(1 & __util$getparams(argc, argv, s_opts)) )
        return  $LOG_FAO(STS$K_ERROR, "bad options");

    __util$showparams(s_opts);

    return  0;
}
```

Run with a file `x.conf` containing

```
! sample
-name=router7   ! the node
-cou=0x20
/server=10.1.2.3
-port=5000
-nosuch=1
```

as `./opts --config=x.conf --trace --password=sekrit -big=99999999999` it prints (from the `%` on, in an `NDEBUG` build; the program defined no `__FAC__`, so its records say `UTILS`)

```
%UTILS-W: x.conf :6 : Skip unrecognized or unused option 'nosuch'
%UTILS-I: name[0:7] ='router7'
%UTILS-I: password[0:6] ='<password>'
%UTILS-I: trace = ON
%UTILS-I: count = 32 (0x00000020)
%UTILS-I: big = 99999999999 (0x000000174876E7FF)
%UTILS-I: port = 5000 (0x1388)
%UTILS-I: server = 10.1.2.3
```

Note `-cou` for `count`: in a file, or on the command line, a unique abbreviation will do. The unknown `nosuch` is a warning in the log, but the status of the call is still a success (`STS$K_INFO`, odd). Add a line `bogus` to the file, which begins with neither `-` nor `/`, and the call fails:

```
%UTILS-E: x.conf :7 : Option must be started with a '-' or '/', skip : 'bogus'
%UTILS-E: bad options
```

The rest of the file is read all the same.

The rules, as the code has them:

- **On the command line** an option starts with `-`, `--` or `/`, and its value follows an `=`: `--port=5000`. The name is taken in any case, and may be shortened to a unique prefix: `--ser` for `server`. A name spelled out in full wins, even if it is also the beginning of a longer one (`name` and `nameserver`). An abbreviation that fits two options (`--co` for `config` and `count`) is an error, logged, and the option is skipped. A name *longer* than the option is not a match, so `NSERVERXYZ` does not select `NSERVER`.
- **The status.** `__util$getparams` returns `STS$K_SUCCESS` when every argument was taken. It returns `STS$K_ERROR` for a bad value, an unknown or ambiguous option, a token that

does not begin with `-` or `/`, or a failure in a configuration file. Each fault is logged through `$LOG_FAO`, and the rest of the arguments is still processed, so one run reports all of them. (Until 1.7.0 the call returned success after all of these.) The array `argv` is never modified. An empty argument is skipped.

- **In a configuration file** each line is `-name=value` (or `/name=value`). A `!` starts a comment, unless it stands inside double or single quotes. Blank lines are skipped. The same rules of abbreviation apply. A line over 254 octets is dropped as a whole, with an error (unless what went beyond is a comment); its tail is not read as a line of its own. An option that is not in the table is logged as a warning and skipped, because one file may serve several programs; it gives the status `STS$K_INFO`, which has the low bit set, so `1 & status` still says success. `OPTS$K_CONF` inside a file is refused with an error: a file does not name another file. When `__util$readconfig` is called by itself, the same statuses are its return value; the worst one of the whole file wins (an error over an info).
- **Quotes.** A value that begins and ends with the same quote, `"a b"` or `'a b'`, loses them, in a file and on the command line (`/name="a b"` when the shell passed the quotes). A lone or unbalanced quote stays in the value. It is applied to every type (`"42"` is the number 42).
- **A switch** (`OPTS$K_OPT`) is set to 1 by its presence. There is no way to switch it off from the line; give the default a value that is right without the switch.
- **There is no “required”.** If an option must be given, the program tests the variable afterwards.
- **Strings go in an `ASC`**, so a value is cut at 254 octets, and the terminating zero is always there.
- `__util$showparams` logs the whole table at `STS$K_INFO`, the values in their types, and shows `<password>` for a password.

`__util$readparams` was declared in the header for years and never defined. It is gone from the header in 1.7.0; use `__util$getparams`.

Chapter 8. The DCL-Style Command Line

8.1 The Idea

On OpenVMS, a command language definition said once what a command looks like - its verb, its parameters, its qualifiers and what type each takes - and the system did the parsing, the checking and the help. A program only asked: *was `/LOG` given? what is the value of P1?* `cli_routines` brings that idea to C with tables in place of a CLD file:

```
verb [sub-verb] [p1 p2 ... p8] [/qualifier[=value] | --qualifier[=value] ...]
```

It works from `argv`. There is no compiler of definitions: you write the tables as C initializers. A table of verbs is an array of `CLI_VERB` ended by an entry with an empty name; the parameters and qualifiers of a verb are arrays of `CLI_PQDESC`, ended the same way.

8.2 The Tables

```
typedef struct __cli_pqdesc__ {
    ASC     name;       /* the name */
    unsigned short  type;       /* CLI$K_FILE, CLI$K_NUM, ... */
    unsigned char   pn;     /* CLI$K_P1 ... CLI$K_P8 for a parameter, CLI$K_QUAL for a
     ↪  qualifier */
    unsigned char   flag;       /* CLI$M_NEGATABLE, CLI$M_LIST, CLI$M_REQUIRED */
    ASC     defval;     /* default value text, or empty */
    CLI_KEYWORD *kwd;       /* keywords, for CLI$K_KWD */
} CLI_PQDESC;

typedef struct __cli__verb__ {
    ASC     name;
    struct __cli__verb__ *next; /* sub-verbs, or NULL */
    CLI_PQDESC  *params, *quals;
    int     (*act_rtn) (struct __cli_ctx__ *clictx, void *act_arg);
    void        *act_arg;
} CLI_VERB;
```

Value types

Constant	Accepts
`CLI$K_FILE`	Any text; not checked
`CLI$K_NUM`	An unsigned number, decimal, `0x` or octal; the whole token must be digits (no minus sign, no trailing text) and must not overflow
`CLI$K_DATE`	`dd-mm-yyyy`, optionally followed by a separator (`-`, a blank or `T`) and `hh:mm[:ss]`; the day is checked against the month, leap years included, and nothing may follow
`CLI$K_IPV4`, `CLI$K_IPV6`	An address that `inet_pton` accepts
`CLI$K_OPT`	No value: a switch
`CLI$K_QSTRING`	Any text; not checked
`CLI$K_UUID`	The canonical 36-character form
`CLI$K_DEVICE`	A name under `/dev`, which must exist
`CLI$K_KWD`	One of the words of `kwd`, or an unambiguous prefix of one. The parse stores the **full** word of the table, not what was typed
`CLI$K_TIME`	An absolute or delta time, checked by `__util$bintim` (Chapter 10)
`CLI$K_MAC`	Six hex octets with `:` or `-` between

Flags

Flag	Meaning
`CLI$M_NEGATABLE`	The qualifier has a `/NOname` form
`CLI$M_LIST`	It takes a list, `/tag=(a,b,c)` or a repeat
`CLI$M_REQUIRED`	It must be given, else the parse fails (parameters too)

A parameter is optional unless it is `CLI$M_REQUIRED`.

8.3 A Complete Program

```
#define __MODULE__  "COPYT"
#define __FAC__     "COPYT"

#include    <stdlib.h>
#include    "utility_routines.h"
#include    "starlet.h"
#include    "cli_routines.h"

static  CLI_KEYWORD s_modes [] = {
    { .name = {$ASCINI("FAST")}, .val = 1 },
    { .name = {$ASCINI("SAFE")}, .val = 2 },
    { .name = {0} }
    };

static  CLI_PQDESC  s_params [] = {
    { .name = {$ASCINI("source")}, .type = CLI$K_FILE, .pn = CLI$K_P1, .flag =
     ↪  CLI$M_REQUIRED },
    { .name = {$ASCINI("target")}, .type = CLI$K_FILE, .pn = CLI$K_P2 },
    { .name = {0} }
    };

static  CLI_PQDESC  s_quals [] = {
    { .name = {$ASCINI("log")},    .type = CLI$K_OPT, .pn = CLI$K_QUAL, .flag =
     ↪  CLI$M_NEGATABLE },
    { .name = {$ASCINI("blocks")}, .type = CLI$K_NUM, .pn = CLI$K_QUAL, .defval =
     ↪  {$ASCINI("512")} },
    { .name = {$ASCINI("mode")},   .type = CLI$K_KWD, .pn = CLI$K_QUAL, .kwd = s_modes },
    { .name = {$ASCINI("tag")},    .type = CLI$K_QSTRING, .pn = CLI$K_QUAL, .flag =
     ↪  CLI$M_LIST },
    { .name = {0} }
    };

static  int s_do_copy   (CLI_CTX *a_ctx, void *a_arg)
{
ASC     l_val;
CLI_KEYWORD *l_kwd;
unsigned    l_state;

    if ( 1 & __cli$get_value(a_ctx, &s_params[0], &l_val) )
        $LOG_FAO(STS$K_INFO, "source = !AD", (unsigned) l_val.len, l_val.sts);

    if ( 1 & __cli$present(a_ctx, &s_params[1], NULL) )
        if ( 1 & __cli$get_value(a_ctx, &s_params[1], &l_val) )
            $LOG_FAO(STS$K_INFO, "target = !AD", (unsigned) l_val.len, l_val.sts);

    if ( 1 & __cli$get_value(a_ctx, &s_quals[1], &l_val) )      /* has a default */
        $LOG_FAO(STS$K_INFO, "blocks = !AD", (unsigned) l_val.len, l_val.sts);

    if ( 1 & __cli$present(a_ctx, &s_quals[0], &l_state) )
        $LOG_FAO(STS$K_INFO, "log is !AZ", (l_state & CLI$M_NEGATED) ? "negated" : "on");

    if ( 1 & __cli$get_keyword(a_ctx, &s_quals[2], NULL, &l_kwd) )
        $LOG_FAO(STS$K_INFO, "mode = !AC (value !SL)", &l_kwd->name, l_kwd->val);

    if ( 1 & __cli$present(a_ctx, &s_quals[3], NULL) )
        while ( 1 & __cli$get_value(a_ctx, &s_quals[3], &l_val) )
```

```
            $LOG_FAO(STS$K_INFO, "tag = !AD", (unsigned) l_val.len, l_val.sts);

    return  STS$K_SUCCESS;
}

static  CLI_VERB    s_verbs [] = {
    { .name = {$ASCINI("copy")}, .params = s_params, .quals = s_quals, .act_rtn =
     ↪  s_do_copy },
    { .name = {0} }
    };

int main    (int argc, char *argv[])
{
void    *l_ctx;
int l_sts;

    l_sts = __cli$parse(s_verbs, CLI$M_OPSIGNAL, argc - 1, argv + 1, &l_ctx);

    if ( l_sts == STS$K_WARN )          /* help was asked for, and has been printed */
        return  EXIT_SUCCESS;

    if ( 1 & l_sts )
        {
        l_sts = __cli$dispatch(l_ctx);
        __cli$cleanup(l_ctx);
        }

    return  (1 & l_sts) ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

Some runs of it:

```
$ copyt cop a.dat b.dat /nolog "--blocks=0x10" "/tag=(x,y)" /mode=sa
%COPYT-I: source = a.dat
%COPYT-I: target = b.dat
%COPYT-I: blocks = 0x10
%COPYT-I: log is negated
%COPYT-I: mode = SAFE (value 2)
%COPYT-I: tag = x
%COPYT-I: tag = y

$ copyt copy /blocks=12
%CLI_RTNS-F: Missing P1 - source !

$ copyt copy a.dat --blocks=12x
%CLI_RTNS-E: Value '12x' is not a valid number

$ copyt copy a.dat b c
%CLI_RTNS-F: Too many parameters, surplus token 'c'

$ copyt copy a.dat /blocks 5
%CLI_RTNS-F: Value '5' must follow '=' of the qualifier '/blocks' (write /blocks=5)

$ copyt copy a.dat --bogus
%CLI_RTNS-F: Unrecognized qualifier 'bogus'

$ copyt copy --help
Usage: copy source [target] [/qualifier ...]
Parameters:
```

```
    P1  source         FILE (./filename.ext, device:\path\filename.ext)
    P2  target         FILE (./filename.ext, device:\path\filename.ext)
Qualifiers:
    /log            OPTION (no value), /NO<name> accepted
    /blocks         DIGIT (decimal, octal, hex), default=512
    /mode           KEYWORD
    /tag            ASCII string in double quotes, list
```

(Built with `-DNDEBUG`; records are shown from the `%` on, without the date, time and thread columns. Note that `cop` was enough for `copy`, and that `/mode=sa` was stored as the full word `SAFE`.)

8.4 The Routines

```
int  __cli$parse     (CLI_VERB *verbs, int opts, int argc, char **argv, void **clictx);
int  __cli$dispatch  (CLI_CTX *clictx);
int  __cli$get_value (CLI_CTX *clictx, CLI_PQDESC *pq, ASC *val);
int  __cli$present   (CLI_CTX *clictx, CLI_PQDESC *pq, unsigned *state);
int  __cli$cleanup   (CLI_CTX *clictx);
int  __cli$get_keyword (CLI_CTX *clictx, CLI_PQDESC *pq, const ASC *val, CLI_KEYWORD
 ↪  **kwd);
int  __cli$help      (CLI_VERB *verbs, CLI_VERB *verb);
void __cli$show_verbs(CLI_VERB *verbs, int level);
void __cli$show_ctx  (CLI_CTX *clictx);
```

__cli$parse matches `argv[0]` against the verb table (the program name is *not* part of it: pass `argv + 1`), then the sub-verb, the parameters and the qualifiers. It builds a context in `*clictx`, which must be released with `__cli$cleanup`, and returns a condition value. `opts` combines `CLI$M_OPSIGNAL` (log the reason of a failure) and `CLI$M_OPTRACE` (trace, in a Debug build of the library). On a failed parse, and after help, `*clictx` is NULL and the half-built context is freed, but the documented idiom of calling `__cli$cleanup` on it anyway is safe.

The result is `STS$K_SUCCESS`; `STS$K_FATAL` for a bad command (an unknown or ambiguous verb or qualifier, a missing required item, a surplus word) or `STS$K_ERROR` for a value of the wrong type; or **`STS$K_WARN` when the user asked for help**, which is printed, and nothing is to be dispatched. Help is not a failure, and a program has to treat it so:

```
if ( l_sts == STS$K_WARN )
    return  EXIT_SUCCESS;       /* the help was printed */
```

Help is recognized in any token: `/HELP`, `-HELP`, `--help`, `-h`, in any case. A bare `help` is an ordinary word - it may be your own verb, or a file named so. With help asked before the verb, the list of commands is printed; with a verb, its parameters and qualifiers, with their types, defaults and flags; with a container verb (`show --help`), its sub-verbs, and `show statistics --help` reaches the sub-verb itself. The text goes through `__util$out`.

__cli$get_value returns the text of a parameter or qualifier in an `ASC`. If the item is absent and has a `defval`, you get the default. For a `CLI$M_LIST` qualifier each call returns the next element, and the call after the last returns `STS$K_ERROR`. For an absent item without a default it returns `STS$K_ERROR` too; an item that is present with an empty value gives `STS$K_WARN`. An absent item is not an error of the program, so the call **never logs it**, with `CLI$M_OPSIGNAL` or without: probing is free. (A NULL context or descriptor is a programming error and is logged.) A keyword value comes back as the **full word of the table**, whatever abbreviation was typed.

__cli$get_keyword(clictx, pq, val, &kwd) goes one step further and hands back the table entry of a `CLI$K_KWD` item - its full name in `kwd->name` and its number in `kwd->val`. Pass the value you got from `__cli$get_value`, or NULL for the first value of the item (or its default). It does not move the cursor of a list. It returns `STS$K_SUCCESS`; `STS$K_WARN` if the item is present with an empty value (`/mode`, `/mode=`), as `__cli$get_value` says it; or `STS$K_ERROR` if the item has no keyword table, is absent without a default, or the value is not a word of the table; nothing is logged. There is no need for `__util$lookup_key` on a stored keyword any more.

__cli$present says whether an item was given: `STS$K_SUCCESS` if so (`*state` carries `CLI$M_PRESENT`, and `CLI$M_NEGATED` for a `/NO` form), `STS$K_WARN` if not. If a qualifier is repeated, the *last* occurrence decides: `/LOG /NOLOG` is negated and `/NOLOG /LOG` is on.

__cli$dispatch calls the `act_rtn` of the deepest verb matched, with the context and `act_arg`, and returns what it returns. An action routine has the signature `int (CLI_CTX *, void *)`.

8.5 Rules of the Parser

- **A verb or qualifier may be abbreviated to its unique prefix**, in any case. A name spelled out in full wins over a longer one it begins: `/LOG` against `/LOGFILE`, and, since 1.7.0, `SET` against `SETUP`. An abbreviation that fits two verbs (`se`) is an error. Verbs that share the same action, parameters, qualifiers and sub-verbs are synonyms, and are never ambiguous with each other.
- **Surplus words are an error.** A word that no parameter slot takes is reported as `Too many parameters`, also for a verb that has no parameters at all (it used to be ignored there). A verb that wants a sub-verb and gets none says `Too few arguments`.
- **A qualifier is `/name` or `--name`.** A single `-name` is not a qualifier. A value follows an `=`: `/blocks=12`. A space-separated value (`/blocks 12`) is not understood; `12` would be a surplus token.
- **`/NOname` is understood** for qualifiers declared `CLI$M_NEGATABLE`. A negated qualifier has no value.
- **Lists:** `/tag=(a,b,c)` or `/tag=a,b,c`, for `CLI$M_LIST`.
- **A path is a value, but only where a path is wanted.** A token that names a defined qualifier stops the positional scan and is left to the qualifiers. A token that begins with `/` and names none is taken as the value of a pending parameter only if that parameter is `CLI$K_FILE` or `CLI$K_DEVICE` (`/etc/passwd` for `source`); for any other parameter, or when all the slots are full, it is an `Unrecognized qualifier` error. An unknown `--name` is always an error: `copy a.dat --bogus` fails, and no longer makes `--bogus` the `target`.
- **`/blocks 5` is an error with a message.** A value that follows a qualifier after a blank is not taken as its value, and the parser says `Value '5' must follow '=' of the qualifier '/blocks'`. Before 1.7.0 the message was a plain `Unrecognized token`.
- **Values are checked strictly.** `CLI$K_NUM` is an unsigned number whose every character is a digit and which fits; `CLI$K_DATE` checks the day against the month. `-5`, `12x` and `31-02-2026` are refused.
- **Sub-verbs** (`SHOW STATISTICS`): the `next` of a verb points to a table of its sub-verbs, which follow the verb immediately; qualifiers come after.
- **Up to eight parameters**, `CLI$K_P1` to `CLI$K_P8`.
- **Values are taken as strings**: the type is a check, not a conversion.
- `cli_routines.h` needs `ASC`. Include `utility_routines.h` **before** it (`starlet.h` brings it in).

8.6 Where It Differs from DCL

There is no CLD compiler and no command tables in a file. There is no prompting for a missing parameter, no `/QUALIFIER` that takes a different default when present than when absent, no `CLI$_NEGATED` condition code (`__cli$present` reports negation in `*state`), no `LIB$GET_INPUT`. The parser is the part of DCL that a Linux program most often needs: the verb, the parameters, the qualifiers and the help. That is what it does.

Chapter 9. Queues and Locks

9.1 The Heritage

The VAX had instructions for doubly linked queues - INSQUE and REMQUE, and their interlocked relatives INSQHI, REMQTI and the rest. A queue was an everyday data structure of an operating system, and a program on VAX built on it without a thought. StarLet gives Linux a queue of the same kind, though not the same instructions: a doubly linked list with a head, a tail, a counter and a **spin lock**. It is not lock-free, and it makes no claim to be.

9.2 The Types

```
typedef struct __entry {
    struct __entry  *left, *right;  /* neighbours; NULL at an end */
    void        *queue;     /* the queue the entry is in, or NULL */
} ENTRY;

typedef struct __queue {
    ENTRY   *head, *tail;
    int lock;           /* on Windows, an SRWLOCK */
    unsigned count;
} __QUEUE;

#define QUEUE_INITIALIZER { (ENTRY *) 0, (ENTRY *) 0, 0, 0 }
```

An `ENTRY` must be the **first member** of whatever you queue:

```
typedef struct item {
    ENTRY   links;          /* first */
    int n;
} ITEM;
```

The `queue` field of an entry must be **zero** before its first insertion. Static and zeroed storage is; memory from `malloc` is not, unless you clear it.

9.3 The Operations

All return a condition value, and all take the lock around the change.

Macro	Does
`$INSQTAIL(que, ent, &count)`	Puts the entry at the tail; `count` gets the number of entries *before*
`$INSQHEAD(que, ent, &count)`	Puts it at the head
`$REMQHEAD(que, &ent, &count)`	Takes the head entry into `ent`
`$REMQTAIL(que, &ent, &count)`	Takes the tail entry
`$REMQENT(que, ent, &count)`	Takes out a given entry from wherever it is; `count` gets the number before
`$CLRQUE(que, &count)`	Empties the queue, detaching every entry; `count` gets the number there were
`$MOVQHEAD(que, ent)`, `$MOVQTAIL(que, ent)`	Move an entry of the queue to the head or the tail; `ent` is the entry itself

```
__QUEUE l_que = QUEUE_INITIALIZER;
ITEM    l_item [3], *l_p;
unsigned l_count;

for ( int i = 0; i < 3; i++ )
    {
    l_item[i].n = i;
    l_item[i].links.queue = NULL;
    $INSQTAIL(&l_que, &l_item[i], &l_count);
    }

if ( 1 & $REMQHEAD(&l_que, &l_p, &l_count) && l_count )
    $LOG_FAO(STS$K_INFO, "took item !UL, there were !UL", l_p->n, l_count);
```

Things to know, all of them checked by running (and, since 1.7.0, by `test_queue`, which also walks a random sequence of every operation against a plain array):

- **An empty queue is a success.** `$REMQHEAD` and `$REMQTAIL` on an empty queue return `STS$K_SUCCESS`, set `ent` to NULL and `count` to zero. Tell "got an entry" from "nothing there" by `ent` (or `count`). Before 1.7.0 `ent` was left as it was, and `$CLRQUE` left `count` unset. `$CLRQUE` on an empty queue reports zero.
- **`$REMQENT` of an entry that is not on the queue** - an empty queue included - is `STS$K_ERROR`: there is nothing to remove. This differs from the removal at an end, where an empty queue is normal.
- **An entry already on this queue** is a success and is left where it is; `count` is not set. An entry on *another* queue is refused with `UTIL$S_INQUE` by both `$INSQTAIL` and `$INSQHEAD`; a lock that could not be taken is `UTIL$S_NOLOCK`; a NULL argument is `UTIL$S_INVARG`. (`$INSQHEAD` returned a plain `STS$K_ERROR` for all three before 1.7.0.)
- **`$MOVQHEAD` and `$MOVQTAIL` work.** They re-point the head, the tail and the neighbours' links; moving the entry that is at the end already is a success and changes nothing; an entry that is not on this queue, or a NULL argument, is `STS$K_ERROR`. The routines take the entry as `void *` (they were declared with `void **ent`, which did not agree with what the macros pass).
- **The lock** is a spin lock. `$LOCK_LONG` (`__util$lockspin`) tries about four billion times and gives up with `STS$K_ERROR`, so it does not hang forever; `$UNLOCK_LONG` releases it. It is not recursive. Use it for short critical sections.

9.4 Atomics

- `$ATOMIC_INC(v)`, `$ATOMIC_DEC(v)` - the GCC `__sync_fetch_and_add` and `_sub` by one; they return the old value.
- `$CMP_STORE_LONG(addr, old, new)` - compare-and-swap of an `int`; true if the store was made. A distant cousin of the Alpha `LDL_L`/`STL_C` pair.

These are full barriers. Use them for a counter and a flag; for anything bigger, use the queue or a real lock.

Chapter 10. Time

10.1 Binary Time Here

On OpenVMS a binary time is a quadword that counts 100-nanosecond ticks from 17-NOV-1858, and a *delta* time is the same quadword negative. StarLet keeps the idea and drops the unit. Its binary time, `fao_time_t`, is **seconds of the UNIX epoch** (a `time_t`), and a delta time is a **negative** number of seconds. That is exactly what the FAO directives `!%T` and `!%D` read, so the formatter and the converters agree and a value goes round the trip.

Routine	Does	After
`__util$gettim(&t)`	The current time	`SYS$GETTIM`
`__util$bintim(text, &t)`	ASCII to binary, absolute or delta	`SYS$BINTIM`
`__util$asctim(&len, &dsc, &t, flag)`	Binary to ASCII; `t` NULL means now	`SYS$ASCTIM`
`__util$add_times(&a, &b, &r)`	Sum	`LIB$ADD_TIMES`
`__util$sub_times(&a, &b, &r)`	Difference	`LIB$SUB_TIMES`

All return condition values. Hundredths of a second are accepted and then dropped, and printed as `.00`. The display routines and the FAO directives read the sign of the number the same way: positive is an absolute time, negative a delta, and **zero means "now"** - so the epoch itself cannot be displayed, and a time before 1970 cannot be written as an absolute time at all. The time zone is a constant, `FAO$K_TZOFFSET`, zero (UTC) unless `starlet.c` is built with another; this is the zone of `bintim`, `asctim`, `!%D` and `!%T` alike.

10.2 The Text

`__util$bintim` accepts, with blanks allowed around:

Form	Meaning
`dd-MMM-yyyy[ hh:mm[:ss[.cc]]]`	An absolute time; the time of day defaults to 00:00:00
`hh:mm[:ss[.cc]]`	Today at that time
`-MMM-` or `dd-MMM-`	The missing date fields come from today
`dddd hh:mm[:ss[.cc]]` or `dddd-hh:mm...`	A **delta** time; the result is negative
`::10`	A delta of ten seconds: fields of the time may be left out, as in DCL

The month is three letters in any case. The day is checked against the month, leap years included. A malformed or out-of-range string gives `FAO$K_IVTIME` (even, so a failure), and the result is left alone. So does an absolute date before 01-JAN-1970: it would come out negative, and every consumer would read it as a delta (until 1.7.0 it did exactly that). The epoch itself gives 0. `__util$asctim` prints an absolute time as `dd-MMM-yyyy hh:mm:ss.00`, the day blank-padded, and a delta as `dddd hh:mm:ss.00`; `flag = 1` asks for the time of day alone.

10.3 Arithmetic

The rules are those of LIB$ADD_TIMES and LIB$SUB_TIMES, with deltas negative:

Operation	Result
delta + delta	delta
absolute + delta	absolute, later
absolute + absolute	FAO$K_IVTIME (at least one must be a delta)
absolute - absolute	delta; FAO$K_NEGTIM if the first is the earlier
absolute - delta	absolute, earlier; FAO$K_NEGTIM if it would go below zero
delta - delta	delta; FAO$K_NEGTIM if the second is the larger
delta - absolute	FAO$K_IVTIME

```
#define __MODULE__  "TIMEX"
#define __FAC__     "TIMEX"

#include    "utility_routines.h"
#include    "starlet.h"

int main    (void)
{
fao_time_t  l_t, l_d, l_r;
char        l_buf [64];
fao_desc_t  l_dsc = $FAO_DESC(l_buf);
unsigned short  l_len;

    __util$bintim("31-OCT-2026 12:30:00", &l_t);
    __util$bintim("1 02:00:00", &l_d);          /* one day and two hours */

    __util$add_times(&l_t, &l_d, &l_r);
    __util$asctim(&l_len, &l_dsc, &l_r, 0);         /*  1-NOV-2026 14:30:00.00 */
    $LOG_FAO(STS$K_INFO, "later: !AD", (unsigned) l_len, l_buf);

    __util$sub_times(&l_r, &l_t, &l_r);         /* absolute - absolute = delta */
    __util$asctim(&l_len, &l_dsc, &l_r, 0);         /* 1 02:00:00.00 */
    $LOG_FAO(STS$K_INFO, "difference: !AD", (unsigned) l_len, l_buf);

    return  0;
}
```

10.4 The Other Time Routines

The header `utility_routines.h` also has three small helpers on `struct timespec`: `__util$add_time` (sum, carrying the nanoseconds), `__util$sub_time(&t1, &t2, &result)` (the difference `t1 - t2`) and `__util$cmp_time(&t1, &t2)`. The difference is normalised since 1.7.0: `tv_nsec` is always in 0 to 999999999, and a negative difference of the nanoseconds borrows a second (it used to leave a negative `tv_nsec`). The comparison returns the sign of `t1 - t2`: negative if `t1` is earlier, zero if equal, positive if later - in practice -1, 0 or 1, but test only the sign. It compares the seconds and does not subtract them, so it cannot overflow or flip its sign for times far apart; both take `const` pointers. And `__util$timbuf` breaks a `time_t` into a `struct tm` with the month and year already adjusted (1 to 12, full year). The log prefix does not use any of them; it has its own cached rendering.

Chapter 11. The TLV Protocol

11.1 The Idea

`avproto` is a way to put typed values in a packet and take them out again, byte-order safe. A packet (a PDU) is a header and a run of TLVs: tag, length, value.

```
PDU:  | header 20 octets | TLV | TLV | ... |

header: | magic 8 | length 4 | csr 4 | seq 4 |            all in network byte order

TLV:    | type 4 bits + id 12 bits | length 16 bits | value 0..length octets |
```

- `magic` is a signature of up to eight octets, zero-padded; the receiver checks it.
- `length` is the number of octets of TLVs that follow the header.
- `csr` is a command or status word and `seq` a sequence number, yours to use.
- A tag's **type** says how the value is read: `TAG$K_BBLOCK` (0) octets, `TAG$K_WORD` (3) 16 bits, `TAG$K_LONGWORD` (4) 32 bits, `TAG$K_QWORD` (5) 64 bits, `TAG$K_UUID` (8) 16 octets. Words are converted between network and host order for you.
- The **id** is yours, 0 to 4095.

Include `utility_routines.h` before `avproto.h`; it needs `STS$K_*`. Since 1.7.0 `avproto.h` no longer switches off three compiler warnings (`-Wparentheses`, `-Wdate-time`, `-Wunused-variable`) for every file that includes it.

11.2 The Routines

```
inline static int __tlv$hset (void *pdu, const unsigned char *sig, unsigned csr, unsigned
 ↪  seq);
inline static int __tlv$hget (void *pdu, const unsigned char *sig, unsigned *len,
 ↪  unsigned *csr,
                  unsigned *seq);
int __tlv$put    (void *pdu, unsigned pdusz, unsigned tag, unsigned type, void *val,
          unsigned valsz);
int __tlv$get    (void *pdu, unsigned pdusz, int *context, unsigned tag, unsigned *type,
          void *val, unsigned *valsz);
int __tlv$lookup (void *pdu, unsigned pdusz, int *context, unsigned tag, unsigned *type,
          void **val, unsigned *valsz);
void __tlv$dump  (void *pdu, unsigned pdusz);
```

`__tlv$hset` initializes the header and sets the length to zero. The signature is a string of 1 to 8 octets, stored zero-padded; a longer one, or NULL, is `STS$K_ERROR` and the PDU is not touched (it used to be cut silently). `__tlv$hget` checks the signature - the same rule - and returns the length, `csr` and `seq` in host order; on a mismatch it returns `STS$K_ERROR` and writes **none** of the three outputs (it used to put `STS$K_ERROR` into `*csr`).

`__tlv$put` appends one TLV to the buffer of size `pdusz` and updates the header's length. The free space is checked against the real length of the record - 2, 4, 8 or 16 octets for a word, longword, quadword or UUID, `valsz` for a block - so it never writes beyond the buffer, and the PDU grows by exactly the record plus its four octets of tag and length. `valsz` for a fixed-size type must be *at least* the size of the type (only that many octets are read); zero is no longer accepted. (Since X.94-06 the size was "optional" for these types, which is how a zero or a short size used to overrun the buffer.) Everything that does not fit - a tag id above 4095, a type above 15, a buffer too small, a corrupt header length, a short `valsz` - is `STS$K_ERROR`, logged, and nothing is written. The size `pdusz` is an `unsigned` now, no longer an `unsigned short`, so a buffer over 64 KB can be used.

`__tlv$get` finds a tag and **copies** the value into your buffer, converting numbers to host order. The length on the wire of a fixed-size record must equal the size of its type; a record of 3 octets tagged as a longword is `STS$K_ERROR`, and so is a UUID shorter than 16. `__tlv$lookup` returns the **address** of the value inside the PDU, unconverted, which suits big octet blocks; it checks the wire length of a fixed-size record as `__tlv$get` does (since 1.7.1; before, it handed out a pointer to a short record), and a wrong one is `STS$K_ERROR` with no pointer. Both validate every record against the real size `pdusz` of what was received, and do not trust the length in the header: a record that runs past the end of the PDU is `STS$K_ERROR`. Both return `STS$K_WARN` if the tag is not there. For a block, UUID or unknown type `valsz` is required: in, the size of your buffer; out, the octets copied.

The `context` makes the routines iterate: start with `-1`; each call leaves it after the TLV it found, so a second call with the same tag finds the next TLV with that tag.

```
#include    <stdio.h>
#include    "utility_routines.h"
#include    "avproto.h"

int main    (void)
{
unsigned char   l_pdu [128];
unsigned short  l_w = 0x1234;
unsigned    l_type, l_len, l_csr, l_seq, l_vlen;
int     l_ctx = -1;
char        l_buf [32];

    __tlv$hset(l_pdu, (const unsigned char *) "DEMOPDU", 1, 7);
    __tlv$put(l_pdu, sizeof(l_pdu), 1, TAG$K_WORD, &l_w, sizeof(l_w));
```

```
    __tlv$put(l_pdu, sizeof(l_pdu), 2, TAG$K_BBLOCK, "hello", 5);

    if ( 1 & __tlv$hget(l_pdu, (const unsigned char *) "DEMOPDU", &l_len, &l_csr, &l_seq)
     ↪  )
        {
        l_vlen = sizeof(l_buf);
        if ( 1 & __tlv$get(l_pdu, sizeof(l_pdu), &l_ctx, 2, &l_type, l_buf, &l_vlen) )
            printf("tag 2: type %u, %u octets: %.*s\n", l_type, l_vlen, (int) l_vlen,
↪   l_buf);
        }

    return  0;
}
```

(This program uses `printf` because it is an example and a test; real code of ours uses `$LOG_FAO`.)

It prints `tag 2: type 0, 5 octets: hello`. A PDU with these two TLVs is 20 + 6 + 9 = 35 octets long.

11.3 Cautions

- Check the status of `__tlv$put`. A record that does not fit, a tag id of 4096 or more, and a `valsz` below the size of the type are all refused, and the PDU stays as it was.
- A PDU on the wire is hostile until proven otherwise. Give `__tlv$get`, `__tlv$lookup` and `__tlv$dump` the size of the buffer you *received*, not the length the header claims.
- `__tlv$dump` writes the value of a TLV in lines of 64 octets, so a long value no longer overruns its buffer (a value of 256 octets or more used to write one octet too far).
- Fixed-size types are checked by `__tlv$get` and not by `__tlv$lookup`.

Chapter 12. Portability

12.1 What Is Tested Where

Platform	State
Linux x86-64 (glibc)	Everything, in CI and by hand; the primary platform
Linux with musl	CI builds and runs the whole suite, also linked statically, with `-Werror`
Linux ARMv8 (RK33xx/RK35xx boards)	Built and tested; the hardware CRC-32 is used
Windows x64, MinGW-w64	Cross-built from Linux; 22 of the 24 test programs are built and run under Wine (`log_prefix` is Linux-only, and `logrotate` is left to Windows 11, where it passes)
Windows, MSYS2 with GCC 14	Builds
Android (NDK, bionic)	Builds with the NDK; not exercised on a device by the test suite

12.2 The Rules the Code Follows

Nothing beyond libc. On Windows the library links `ws2_32` (for the syslog socket and the address values of the command parser) and sets `_WIN32_WINNT` to 0x0600 for `WSASendMsg`.

No hard dependency on glibc extras. `strings.h` is included explicitly for `strncasecmp` (musl does not export it from `string.h`). The thread number comes from `syscall(SYS_gettid)`, which glibc, musl and bionic all offer; the `gettid()` wrapper is not used because bionic hides it.

Endianness. `avproto` uses `be16toh` and its relatives, and where Windows has no `<endian.h>` the header supplies the `_byteswap_*` stand-ins (Windows is little-endian only here).

Time. The log prefix reads `CLOCK_REALTIME_COARSE` on Linux. On Windows it reads `clock_gettime(CLOCK_REALTIME)` under MinGW-w64 and `timespec_get` otherwise; the fraction is milliseconds there as on Linux (before 1.7.0 Windows printed six digits).

Logging files. On Windows the log file is opened with `FILE_SHARE_DELETE`, so that rotation can rename it while it is open; `ftruncate` is `_chsize`.

CRC-32. The engine is chosen at run time. On ARMv8 it asks `getauxval(AT_HWCAP)` for `HWCAP_CRC32`, so no `-march` flag is needed for the hardware path to be used (the README still tells the reader to add one; the engines are built with target attributes, so the flag is not needed). `STARLET_CRC32=SOFTWARE` forces the tables.

Android. Compile with `-DANDROID_LOGCAT` to have records also sent to `logcat`.

Build type. Release by default, with `NDEBUG`. A build for a device with no tracing needs nothing else.

Chapter 13. The House Style

The sources hold to the manner of the OpenVMS engineers. It is not decoration: a reader who knows the convention can tell the lifetime and the origin of a name at sight, and the next reader, who is the author a year later, will thank him.

13.1 Naming

Form	Meaning
`g_name`	A global
`l_name`	A local
`s_name`	A static: a variable, or a routine not exported from its module
`a_name`	A parameter of a routine
`__util$name`, `__cli$name`, `__tlv$name`	An exported routine, module prefix, `$`, then the name
`s_fao$name`	A static routine: `s_`, module, `$`, name
`FAC$K_NAME`	A constant; `FAC$M_NAME` a mask; `FAC$_NAME` a condition value
`$NAME`	A macro

A `$` in a name is plain C for these compilers, and is the sign by which an OpenVMS programmer reads “this belongs to the facility before the `$`”.

13.2 Formatting

Whitesmiths style: the braces of a nested block stand at the indentation of the statements they enclose, the brace of a routine at column zero. Tabs, eight columns wide. Declarations at the top of a routine, without indentation. One parameter per line in a definition, the closing parenthesis on its own line. A single statement under `if` takes no braces. A routine's return type and name are separated by a tab.

```
int __util$example  (
        int a_count
            )
{
int l_i;

    for ( l_i = 0; l_i < a_count; l_i++ )
        {
        if ( l_i == 3 )
            return  STS$K_SUCCESS;
        }

    return  STS$K_WARN;
}
```

13.3 Identification and Revision

Every module has two names for its version, `__IDENT__` and `__REV__`, defined at its head and agreeing with the modification history:

```
#define __IDENT__   "V.01-34"
#define __REV__     "1.34.0"
```

The rule: `X-aa-bbECOc` becomes `aa.bb.c`. A functional change raises `bb` and clears the ECO; a small correction of a fault keeps `bb` and adds `ECOc`. The present modules are:

Module	IDENT	REV
`utility_routines`	`V.01-34`	`1.34.0`
`starlet`	`X.01-16`	`1.16.0`
`cli_routines`	`X.00-17`	`0.17.0`
`avproto`	`X.94-12`	`94.12.0`

The header `starlet.h` carries an identification of its own, `X.01-04`, revision `1.4.0`. The delivery as a whole has one more number, the `project()` version in `CMakeLists.txt`, 1.7.1. It is independent of the module identifications and rises with every delivery that changes anything, so the name of the package says which delivery it is.

13.4 The Modification History and the Functional Description

Every entry in the history has a date, the author's initials (`RRL` for Ruslan R. Laishev, `CLD` for Claude), the new identification and what changed *and why*. The history is a part of the program; it is the one place where the reasons live.

Every routine opens with a block in the VMS manner:

```
/*
**++
**  FUNCTIONAL DESCRIPTION:
**
**  What the routine does, in plain sentences.
**
**  FORMAL PARAMETERS:
**
**  a_name  What it is, and whether it is input or output
**
**  IMPLICIT INPUTS/OUTPUTS:
**  What it reads or writes besides its parameters.
**
**  RETURN VALUE:
**  The condition values it can give, and what each means.
**--
*/
```

13.5 Condition Values, Not Prints

A routine returns a condition value and delivers its result in an output parameter. Messages and logs go **only** through `$LOG_FAO` (with an explicit `STS$K_*` severity) and `$PUTMSG_FAO` (with a condition value of the catalogue); a thin wrapper over them is fine. No `printf`, `fprintf`, `puts`, `perror`, `syslog` or a bare `write(2)` for a message. The exceptions are the tests, and the display of a help text. This chapter is written that way, and so are the examples of this manual, save the one that says otherwise.

Chapter 14. The Tests

14.1 What Is There

Run them all with `ctest --test-dir build`, or without CMake with `sh tests/run.sh`. Every test prints its own account and exits non-zero on a failure. Each uses the tiny harness of `test_harness.h`: `CHECK(cond, message...)` and `TEST_DONE()`. The tests are exempt from the house rule on messages and may print as they like.

Test	What it guards
`status_convention`	The low bit: every success code odd, every failure even; `bintim`, `asctim` and `CLI$K_TIME` honour `1 & status`; `$FAC/$MSG/$SEV` invert `$DEFFAC/$DEFMSG/$DEFSEV` field for field
`cli_features`	Defaults, negatable qualifiers with “last wins”, value lists, `__cli$present`, required items
`cli_help`	The built-in help detector: `/HELP`, `-h`, `--help` yes, a bare `help` no; `*clictx` zeroed on early failure
`cli_nested`	`SHOW STATISTICS`: a sub-verb is looked up in its parent’s list
`cli_filepath`	`/etc/passwd` stays a value while a real qualifier still parses; a bare surplus token is reported
`tlv`	A full PDU round trip: header, word, longword, quadword, blocks, lookup, a missing tag
`log_overflow`	An oversized record is truncated in bounds and its newline or zero lands inside the buffer; run under ASan
`setlogfd`	The log goes to a given descriptor without seizing the standard streams; a negative descriptor is refused
`log_prefix`	The cached prefix is byte-identical to the old `snprintf`; the thread number equals the kernel’s
`log_fao`	The `$LOG_FAO` envelope, FAO body, address directives, the zero-parameter form
`logrotate`	Below the limit nothing; past it `file.1 ... file.keep`; a fresh file for the next record; `keep=0`; keep over 99 refused; no rotation on `setlogfd`
`putmsg_fao`	A catalogue message reads exactly the parameters it was given; severity-independent lookup; the facility stamp

Test	What it guards
`fao_addr`	`!%M`, `!%I`, `!%6`, `!%U`; RFC 5952 corner cases; parity with `inet_ntop()`
`pattern_match`	Fixed vectors, a 500,000-case fuzz against a dynamic-programming reference (wildcard characters in the subjects too), the `?` alias, the hostile pattern
`base64`	RFC 4648 vectors; a round trip over every length; unpadded input; the rejections; a short buffer still reports the size
`crc32`	Check values against a bitwise reference; every length to 2048 at every alignment; every split point; a 4 MB buffer; run again under `STARLET_CRC32=SOFTWARE`
`lookup_key`	Exact, case-blind and abbreviated lookup: full keyword wins, a unique abbreviation selects, synonyms are not ambiguous, an ambiguous one is `STS$K_WARN`, the entry is inside the table
`fao_edge`	A control string cut short in every directive, in exact-size heap blocks under ASan; a year above 9999; a field over 512 octets; a short parameter list; the `FAO$K_*` codes; zero and delta times; `starlet.h` as the first include
`tlv_hostile`	Overrun attempts of `__tlv$put` for every type, truncated and lying records and wrong wire lengths for `get` and `lookup`, a short UUID, a 300-octet dump, a signature over eight octets, the header length against the records
`cli_rules`	`SET/SETUP`, synonyms, surplus words, help of a sub-verb, the stored keyword and `__cli$get_keyword` (also an empty value), unknown qualifiers, the silent probe, `/q value`, date and number checks
`queue`	The empty-queue rule, the status codes, `remqent` of a stranger, the moves of the first, last and a middle entry, and a random walk of every routine against a plain array
`strings`	`hex2bin`, `bzero`, `fill`, `movc5`, `strelem`, `str2asc`, `strcat_rx`, `collapse`, `uncomment`, `mask2sym/val2sym`, `isinrange2`, the `timespec` helpers, `$ILE3INI` and the CRC-32 check value; exact-size buffers
`params`	`OPTS` sizes and signedness, quotes, abbreviations, the status after errors, an untouched `argv`, 255-octet texts, the configuration file, `showparams`
`log_misc`	The millisecond fraction, severity and syslog of the log routines, the `[mod\func:line]` form, `faohex` with a NULL length, the `[FAO ...]` marker, an undefined severity, the statuses of `inimsg` also from many threads, `syslog` with no host

That is twenty-four programs and twenty-five runs. The seven new tests of 1.7.0 are the

ones that were missing when the first edition of this manual was written: the queues, `__util$bzero` and its neighbours, the options parser, and the corners of FAO, the command line and the TLV routines. Not tested at all: the `__util$out` raw writer on its own (it is exercised through the help text and the log tests). `tests/run.sh` lists every program now; it is the same set as CTest, less the second run of `crc32`.

Appendix A. Quick Reference Card

Include order: `utility_routines.h` (or `starlet.h`, which brings it in), then `cli_routines.h` or `avproto.h`, which need it first. Define `__MODULE__` and `__FAC__` before all of them.

Condition values

Macro	Meaning
`1 & sts`	The test: success if non-zero
`$DEFFAC(n) $DEFMSG(n) $DEFSEV(s)`	Build a condition value from its parts
`$FAC(c) $MSG(c) $SEV(c)`	Take a condition value apart
`$STS_ERROR(c)`	True for ERROR and FATAL
`STS$K_WARN 0, SUCCESS 1, ERROR 2, INFO 3, FATAL 4`	The severities
`STS$M_SYSLOG`	OR into a severity: also to `syslog(3)`, `LOG_INFO` or `LOG_ERR`
`FAO$K_NEGTIM 0x08, BUFOVF 0x0A, IVTIME 0x0C, BADDIR 0x12, NOPRM 0x1A, FLDOVF 0x22`	The completion codes of FAO and time
`UTIL$S_INVARG 0x102, INQUE 0x202, NOLOCK 0x302, DUPFAC 0x400`	The codes of the queues and of `__util$inimsg`

Logging and messages

Macro	Meaning
`$LOG_FAO(sev, ctl, ...)`	A log record; FAO control string; at most 16 parameters; returns the severity
`$LOG(sev, fmt, ...)`	A log record; `printf` format; returns the severity
`$PUTMSG_FAO(sts, ...)`	A catalogue message; returns `sts`
`$TRACE(fmt, ...)`, `$IFTRACE(cond, fmt, ...)`	Debug trace; nothing with `NDEBUG`
`$DUMPHEX(ptr, len)`	Hex dump to the log
`__util$inimsg(&desc)`	Register a catalogue (array must not be `const`); `UTIL$S_INVARG`, `UTIL$S_DUPFAC` on failure
`__util$deflog(file, NULL)`, `__util$setlogfd(fd)`	Choose the log destination
`__util$rotatelogfile(limit, keep)`	Rotate the log file
`__util$syslog(fac, sev, tag, msg, len)`	One UDP datagram to the collector remembered by `__util$deflog` (nothing else sends to it)

FAO: the directives are in Section 5.3; `$FAO_DESC(buf)` is the descriptor initializer.

Strings and data

Macro	Meaning
`$ASCDECL(a, "s")`, `$ASCINI("s")`, `$ASCLEN(p)`, `$ASCPTR(p)`, `$ASC(p)`	Counted strings
`$ARRSZ(a)`, `$MIN`, `$MAX`, `$RANGE`, `$ISINRANGE`	Small helpers
`$BIN2HEX(s, d, n)`	Octets to hex text
`$SYM_REC_INI(NAME)`, `$SYM_REC_EOL`	Rows of a value-to-name table
`$ILE2INI(code, len, ptr)`, `$ILENULL`	Item list element initializer, terminator
`$ILE3INI(item, code, len, ptr, retlen)`	Fills an ILE3 object; a statement

Queues and locks

Macro	Meaning
`QUEUE_INITIALIZER`	Initializer of a `__QUEUE`
`$INSQTAIL $INSQHEAD $REMQHEAD $REMQTAIL $REMQENT $CLRQUE`	The operations; each returns a condition value. An empty queue gives SUCCESS, `ent` NULL, `count` 0
`$MOVQHEAD(que, ent)`, `$MOVQTAIL(que, ent)`	Move an entry of the queue to an end
`$LOCK_LONG(p)`, `$UNLOCK_LONG(p)`	Spin lock on an `int`
`$ATOMIC_INC(v)`, `$ATOMIC_DEC(v)`, `$CMP_STORE_LONG(p, old, new)`	Atomics

Command line

Macro	Meaning
`CLI$K_P1 ... CLI$K_P8`, `CLI$K_QUAL`	Position of an item
`CLI$K_FILE NUM DATE IPV4 IPV6 OPT QSTRING UUID DEVICE KWD TIME MAC`	Value types
`CLI$M_NEGATABLE`, `CLI$M_LIST`, `CLI$M_REQUIRED`	Flags of a descriptor
`CLI$M_PRESENT`, `CLI$M_NEGATED`	State bits from `__cli$present`
`CLI$M_OPSIGNAL`, `CLI$M_OPTRACE`	Options of `__cli$parse`
`__cli$get_keyword(ctx, pq, val, &kwd)`	The table entry of a stored `CLI$K_KWD` value (new in 1.7.0)
`__cli$parse` result	SUCCESS; WARN = help printed, do not dispatch; FATAL = bad command; ERROR = bad value

Keywords, options, TLV: `UTIL$K_LOOKUP_NCASE`, `UTIL$K_LOOKUP_ABBREV`; `OPTS$K_INT STR OPT PORT IP4 CONF PWD` and `OPTS_NULL`; `TAG$K_BBLOCK 0`, `WORD 3`, `LONGWORD 4`, `QWORD 5`, `UUID 8`.

Appendix B. History of the Library

This is taken from the modification history of `CMakeLists.txt`. The number is the `project()` version, which is raised with every delivery that changes a module.

Date	Version	What changed
08-MAR-2024	-	The set is begun, as a set of “utility routines for plain C coders”
24-MAR-2026	-	`__MAIN_FOR_DEBUG__` added for debug and development
01-AUG-2026	-	Install and packaging rules apply to a top-level build only; the set is installable as a CMake package, with `StarLet::starlet`; CPack
07-AUG-2026	1.1.0	The package version becomes the single delivery number
07-AUG-2026	1.1.1	Manuals updated: `PUTMSG_FAO`, the command syntax, `--name`, the unrecognized-token error
07-AUG-2026	1.2.0	Message lookup made independent of severity
07-AUG-2026	1.2.1	Lookup compares facility then message number; the found record takes the caller’s severity
07-AUG-2026	1.2.2	The facility name returns to the record descriptor; the envelope `%facility-S-text`
07-AUG-2026	1.3.0	FAO address directives `!%M`, `!%I`, `!%6`, `!%U`
13-AUG-2026	1.4.0	`!%6` gives the RFC 5952 canonical form; the manuals gain the FAO appendix
13-AUG-2026	1.4.1	Cached log-record prefix, about four times faster logging
13-AUG-2026	1.4.2	`!%6` shows an IPv4-mapped address in the mixed notation
14-AUG-2026	1.4.3	`project()` says `LANGUAGES C`; no C++ compiler needed

Date	Version	What changed
14-AUG-2026	1.4.4	The IDENT/REV rule is written down
14-AUG-2026	1.4.5	House rules across the package; the archive named `<package>-<ident>.tar.gz`
14-AUG-2026	1.4.6	`pattern_match` becomes iterative; the hostile pattern drops from 8.6 s to 318 ns
14-AUG-2026	1.4.7	The musl job in CI
14-AUG-2026	1.4.8	Eight routines audited against the routine template
29-AUG-2026	1.5.0	`$LOG_FAO`
29-AUG-2026	1.5.1	Delivery unpacks into `starlet-<ident>/`
31-AUG-2026	1.5.2	The facility name stamped into each message record at `inimsg` time
01-SEP-2026	1.5.3	No dollar-sign identifiers in CMake files
01-SEP-2026	1.6.0	`?` as an alias of `%` in `pattern_match`
01-SEP-2026	1.6.1	Documentation brought up to date
10-SEP-2026	1.6.2	`base64` and `unbase64`
11-SEP-2026	1.6.3	`$MSG()` fixed so that it inverts `$DEFMSG()`
19-SEP-2026	1.6.4	The mnemonic field of the message record; `%FAC-S-MNEMONIC`
03-OCT-2026	1.6.5	CRC-32 chooses its engine at run time (PCLMULQDQ, ARMv8 CRC32, tables)
03-OCT-2026	1.6.6	Release is the default build type; `NDEBUG` comes with it
04-OCT-2026	1.6.7	`__util$rotatelogfile`
06-OCT-2026	1.6.8	Thread number by `syscall(SYS_gettid)`; the Android NDK builds
20-OCT-2026	1.6.9	GCC 14 on Windows (MSYS2) builds
22-OCT-2026	1.6.10	Windows x64 with MinGW-w64 from Linux; tests under Wine; the log shares delete, so rotation works there
31-OCT-2026	1.6.11	The ABBREV mode of `__util$lookup_key` corrected; `test_lookup_key`; the first edition of the Programmer’s Guide and the Reference Manual (order numbers -0111), written against this delivery, with the list of defects that they found

Date	Version	What changed
31-OCT-2026	1.7.0	The defects of 1.6.11 fixed: `utility_routines` V.01-34 (queues, strings and buffers, options and configuration files, logging and syslog, thread-safe `inimsg`, a new pattern matcher), `starlet` X.01-16 (bounded FAO scanner, `FAO$K_*` condition values, time rules), `cli_routines` X.00-17 (DCL verb matching, strict values, `__cli$get_keyword`), `avproto` X.94-12 (bounds-safe TLV routines), `starlet.h` X.01-04 (includes `utility_routines.h`); seven new tests; both manuals revised (-0170); Appendix D lists the changes
31-OCT-2026	1.7.1	The eight items left from 1.7.0: `__tlv$lookup` checks the wire length; the log routines return `STS$K_ERROR` for an undefined severity; a `[FAO ...]` marker for a bad control string; `__util$hex2bin` validates; `OPTS$K_INT` 1/2/8 unsigned, 4/0 signed; quotes dropped from a value; `__cli$get_keyword` WARN for an empty value; the `__util$deflog` comment; `avproto` X.94-12ECO1, `utility_routines` V.01-34ECO1, `cli_routines` X.00-17ECO1; both manuals revised (-0171)

The modules' own histories go back further: `utility_routines.h` to September 2017, the logging and `$GETMSG` routines to 2017 and 2021, `cli_routines` to August 2018, `avproto` to July 2018.

Appendix C. Known Limitations: What Is Still True

The first edition of this manual carried a long list of defects found in the 1.6.11 sources. They are fixed in 1.7.0, and Appendix D lists what changed. What follows is what is still true of the library, found by reading the sources and by running small programs. Most of it is the library as it was meant to be; the rest are corners. Nothing is hidden by this manual.

By design

- `__util$crc32c` computes the **IEEE 802.3 CRC-32** (check value `0xCBF43926` for `"123456789"`), not the Castagnoli CRC-32C that its name suggests. The name is kept, because programs depend on it, and the header now says what the routine is.
- A binary time of **zero means "now"** in `!%D`, `!%T` and `__util$asctim`, and a **negative time is a delta**. A date before 1970 cannot be written as an absolute time; `__util$bintim` refuses one.
- A qualifier's value must follow an `=`: `/blocks=5`. The form `/blocks 5` is refused, now with a message that says why.
- The **ILE3 layout puts the code first**, then the length; the layout of OpenVMS puts the length first. The structure is a shape for portable code, not a binary format for a system service.
- `__util$fao`, the variable-argument form, cannot know how many parameters it was given. Use the macros, or `__util$faol`, which has the count.
- The **FAO engine logs nothing** and reports a bad control string through its status only. `$LOG_FAO` and `$PUTMSG_FAO` add a `[FAO ...]` marker to the record (since 1.7.1) but keep their return value; they cannot tell the program. Test a message once, by looking at it.
- A severity above `STS$K_FATAL` is printed as `?`; since 1.7.1 the log routines return `STS$K_ERROR` for it. Pass only the five severities.
- `__util$cmpasc` of two counted strings puts a shorter string before a longer one, whatever the letters: it is a test for equality, not a dictionary order.
- An entry that is already on the queue is a success for `$INSQTAIL` and `$INSQHEAD`, and `count` is not set. Test `count` only after a removal.
- The wildcard pattern has no escape: `*`, `%` and `?` cannot be matched literally.
- A switch (`OPTS$K_OPT`) can only be turned on from the command line, and the library cannot tell a program that an option is mandatory. The `ASC` that holds a text value is cut at 254 octets.
- The `[module\routine:line]` part of a log record appears only in a unit compiled without `NDEBUG`. After a `fork()` the child still carries the thread number of its parent in the log prefix, until it execs.
- `$LOG_FAO` and `$PUTMSG_FAO` take at most sixteen parameters, and the body of a record is cut at 2047 octets.
- The library has no message vectors, no condition handlers, no `LIB$GET_INPUT`, no CLD compiler, no `__util$traceback`, and no lock-free queue.

Corners that are not by design

- The README still advises an ARMv8 user to add `-march=armv8-a+crc` for the hardware CRC-32. The engine is chosen at run time and needs no flag.

- `__cli$show_ctx` and `__cli$show_verbs` are debugging aids; their output goes to the log and its layout is not promised.

Appendix D. Changes in Versions 1.7.0 and 1.7.1

Changes in Version 1.7.1

Version 1.7.1 fixes the eight items that the writing of the 1.7.0 manuals left open. The module identifications are `avproto` X.94-12ECO1, `utility_routines` V.01-34ECO1 and `cli_routines` X.00-17ECO1; the package version is 1.7.1. The changes that can make an existing program behave differently are marked **(check)**.

- **`__tlv$lookup` checks the wire length** of a fixed-size record (word 2, longword 4, quadword 8, UUID 16), as `__tlv$get` does. A wrong length is `STS$K_ERROR` and no pointer is given. Before, a pointer to a short record came back as a success **(check)**.
- **An undefined severity returns `STS$K_ERROR`.** `$LOG`, `$LOG_FAO`, `__util$log`, `__util$log_fao`, `__util$logd`, `__util$logd_fao` and `__util$log2buf` still print `?` for a severity of 5, 6 or 7, but return `STS$K_ERROR` (2, low bit clear) instead of `STS$K_UNDEF` (5, low bit set), so `return $LOG(sev, ...)` with a wrong severity no longer reports success **(check)**. Nothing in StarLet depended on the 5. The values 0 to 4, with or without `STS$M_SYSLOG`, are unchanged.
- **A bad FAO status is made visible.** When the formatter of `$LOG_FAO`, `$PUTMSG_FAO` or their debug forms returns a status with the low bit clear, the record ends with `[FAO BADDIR]`, `[FAO NOPRM]`, `[FAO FLDOVF]` or `[FAO BUFOVF]` (`[FAO 0x...]` for another status); a record that fills its buffer is cut to make room for the marker. The record is written, the return value is the same; a program that parses the log will see the marker **(check)**.
- **`__util$hex2bin` validates its input.** An odd length or a character that is not a hexadecimal digit gives **-1**, and nothing is written; the text is checked before the first octet is stored. Before, an odd length produced a leading single-digit octet and a bad character a meaningless value **(check)**. A length of 0 still gives 0. No code in the tree used an odd length.
- **`OPTS$K_INT` has a sign rule.** Sizes 1, 2 and 8 are unsigned: a leading `-` is refused (it used to be stored as `0xFF`, `0xFFFE`, ...) and the range is 0 to 2^n-1. Size 4 and size 0 (`int`) are signed, with the range -2147483648 to 2147483647: a value from 2147483648 to 4294967295 or `0xFFFFFFFF` is refused now (it used to be stored as a negative number) **(check)**. An option that holds a 32-bit mask should be given the size 8. `__util$showparams` prints 1, 2 and 8 unsigned and 4 and 0 signed. The header comment of `OPTS$K_INT` says so.
- **A quoted value loses its quotes.** If a value, after the white space is trimmed, begins and ends with the same quote, `"` or `'`, in a configuration file or on the command line, it is stored without them **(check)**. A lone or unbalanced quote is kept. It holds for every option type; a program that removed the quotes itself will find nothing to remove.
- **`__cli$get_keyword` returns `STS$K_WARN`** for an item that is present with an empty value (`/mode`, `/mode=`), the same as `__cli$get_value`. Absent stays `STS$K_ERROR`. A program that tested `1 & status` is not affected; one that compared with `STS$K_ERROR` is **(check)**.
- **The comment of `__util$deflog` is corrected**: it remembers the syslog host, nothing is sent to it unless the program calls `__util$syslog`; `STS$M_SYSLOG` goes to the local `syslog(3)`.
- Tests: `tlv_hostile` (lookup, 9 checks), `log_misc` (8), `strings` (5), `params` (8) and `cli_rules`

(8) gained checks.

Changes in Version 1.7.0

Version 1.7.0 fixes the defects that were found while the first edition of the manuals was written for 1.6.11, and tightens several rules that were loose. This appendix lists every change that a program using the library could notice. The module identifications are `utility_routines` V.01-34, `starlet` X.01-16, `cli_routines` X.00-17, `avproto` X.94-12 and the header `starlet.h` X.01-04.

The changes that can make an existing program behave differently are marked **(check)**. Look at these first.

Condition values

- The FAO and time codes are real condition values now, all even: `FAO$K_BUFOVF` 0x0A, `FAO$K_IVTIME` 0x0C and `FAO$K_NEGTIM` 0x08 as before, **`FAO$K_BADDIR` 0x12** (it was 6) **(check)**. New: `FAO$K_NOPRM` 0x1A (too few parameters) and `FAO$K_FLDOVF` 0x22 (a field text over 512 octets, cut).
- `UTIL$S_NOLOCK` is **0x302** (it was 0x202, the same as `UTIL$S_INQUE`) **(check)**. `UTIL$S_DUPFAC` 0x400 is new. All four `UTIL$S_*` codes are declared in one place, with the low bit clear.
- `$INSQHEAD` returns the same codes as `$INSQTAIL`: `UTIL$S_INVARG`, `UTIL$S_INQUE`, `UTIL$S_NOLOCK`. It returned a plain `STS$K_ERROR` for all of them.
- `__util$inimsg` returns `UTIL$S_INVARG` for a NULL descriptor and `UTIL$S_DUPFAC` for a facility that is registered already; both had been `STS$K_WARN`. It is thread-safe. The line printed for an unknown condition begins with `%`.
- When a FAO call finds more than one fault, the first one is its status.

Includes and declarations

- `starlet.h` includes `utility_routines.h` itself, and no longer defines fallback values of the severities: the two headers may be included in either order. `cli_routines.h` and `avproto.h` still need `utility_routines.h` first.
- `avproto.h` no longer switches off three GCC warnings for every file that includes it.
- `__util$readparams`, declared but never defined, is removed.
- `__util$movqhead(void *que, void *ent)` and `__util$movqtail(void *que, void *ent)` take the entry itself; they were declared with `void **ent` **(check)**. The macros are unchanged.
- `$ILE3INI(item, code, len, ptr, retlen)` is a statement macro that compiles with ordinary arguments; the object, not a pointer, is its first argument. The layout of the ILE3 (code first) is documented.
- `__util$sub_time` and `__util$cmp_time` take `const` pointers. `__util$pattern_match` names its parameters in the header.
- `__tlv$put` takes the size of the PDU as an `unsigned`; it was an `unsigned short`.
- `CLI_PQDESC.flag` is documented as in use (negatable, list, required); `__cli$show_ctx` is declared; `__cli$get_keyword` is new (Section 8.4).

Logging

- The fraction of the second in a record is true milliseconds, 000 to 999, on every platform **(check)**. It was the nanoseconds shifted by 20 bits (0 to about 953), and six digits on Windows.
- `$LOG`, `$LOG_FAO` and `__util$log2buf` return the severity logged. The syslog bit is removed before the severity is checked, so a flagged call no longer returns 5 **(check)**. The debug forms (`__util$logd`, `__util$logd_fao`) honour the syslog bit, which they ignored.
- The syslog priority is `LOG_INFO` for a severity with the low bit set and `LOG_ERR` otherwise; `LOG_PID` is no longer mixed into the priority. (Every message used to go out as `LOG_INFO`.)
- `__util$syslog` with no collector set is a quiet `STS$K_SUCCESS`: no `perror`, no `abort()`, no message on the standard error. The socket is created once, thread-safely. `__util$deflog` ignores, with a message, a collector address that is not IPv4.
- A debug record with no module name prints `[routine:line]`, not `[(null)\routine:line]`; `$TRACE` prints one backslash, not two.
- `__util$faohex` accepts a NULL `outlen`.

FAO and time

- The scanner never reads past the end of the control string: a string that ends inside a directive (`!`, `!A`, `!%`, `!3*`) is `FAO$K_BADDIR`.
- `!%D`, `!%T` and `__util$asctim` share one renderer. A negative time is a delta (`dddd hh:mm:ss.cc` or `hh:mm:ss.cc`), zero is "now", a year above 9999 is printed in full.
- `__util$bintim` refuses an absolute date before 01-JAN-1970 with `FAO$K_IVTIME` **(check)**. It used to return a negative number, that is, a delta.
- A `!n<...!>` field text over 512 octets is cut with `FAO$K_FLDOVF` (it was cut silently). A control string that wants more parameters than `__util$faol` was given gets zeros and `FAO$K_NOPRM`.
- `__util$cmp_time` returns the sign of the difference (in practice -1, 0, 1) and cannot overflow; `__util$sub_time` returns a normalised `timespec`. `__util$isinrange2` includes the borders **(check)**.

Queues and buffers

- `__util$movqhead` and `__util$movqtail` work; they re-point the head, the tail and the links.
- `$REMQHEAD` and `$REMQTAIL` on an empty queue: `STS$K_SUCCESS`, `*ent` = NULL, `*count` = 0 **(check)**; `$CLRQUE` sets `*count` to 0. `$REMQENT` of an entry that is not on the queue, an empty queue included, is `STS$K_ERROR`.
- `__util$bzero` clears every octet. `__util$iszero` reads no misaligned words and sees every octet where `long` is four bytes. `__util$fill` refuses an empty pattern (it looped for ever).
- `__util$strelem` is right for every index, and leaves `dst` empty when there is no such piece.
- `__util$str2asc` cuts at 254 octets and keeps the terminating zero inside `sts[]`. `__util$strcat_rx` can fill the buffer to `dstsz - 1` and never writes at `dst[dstsz]`; `__util$collapse` no longer needs one octet more than it was given; `__util$mask2sym` counts the separator and the zero when it checks the room.
- `__util$uncomment` skips quoted parts, in single or double quotes **(check)**: a `!` inside a quoted value of a configuration file no longer starts a comment.
- `__util$pattern_match` is written afresh: a `*` in the subject string is an ordinary octet (`"*ba"` matches `"*a"`), a NULL argument is 0.
- `__util$crc32c` and `__util$faohex` are documented as what they are.

Options and configuration files

- `__util$getparams`, `__util$readconfig` and `__util$showparams` share new helpers.
- `OPTS$K_INT` stores exactly 1, 2, 4 or 8 octets as the size says (0 is `int`). A number that does not fit, or has text after it, is a bad value **(check)**.
- A bad value, an unknown option, an ambiguous abbreviation or a stray word makes `__util$getparams` return `STS$K_ERROR`; it used to return success **(check)**. The remaining arguments are still processed.
- A name may be abbreviated to a unique prefix, on the command line as well as in a file; the full name wins. The case does not matter.
- In a configuration file an option that is not in the table is logged as a warning and gives `STS$K_INFO` (low bit set); `OPTS$K_CONF` inside a file is an error; a line over 254 octets is dropped whole.
- `OPTS$K_PORT` is 0 to 65535 in host byte order (as documented); `OPTS$K_STR` stops at 254 octets; `argv` is no longer modified; the messages go through `$LOG_FAO`.

The command line

- A verb spelled out in full wins over a longer one it begins (`SET`, `SETUP`); a unique abbreviation selects; verbs with the same action, parameters, qualifiers and sub-verbs are synonyms.
- A surplus word is an error (`Too many parameters`), also for a verb without parameters. A container verb without a sub-verb says `Too few arguments`.
- An unmatched `/name` is taken as a path only for a pending `CLI$K_FILE` or `CLI$K_DEVICE` parameter; otherwise, and for every unknown `--name`, it is an `Unrecognized qualifier` error **(check)**. It used to become the next optional parameter.

- `/q value` (no `=`) gets a message that names the cure.
- `CLI$K_KWD` stores the full keyword of the table; `__cli$get_keyword` returns its table entry. A program that compared the stored text with the abbreviation typed must be changed **(check)**.
- `CLI$K_NUM` accepts only an unsigned number that fits; `CLI$K_DATE` checks the calendar and takes nothing after the time.
- `__cli$get_value` of an absent item is a quiet `STS$K_ERROR`, with `CLI$M_OPSIGNAL` too; the idiom of clearing `opts` before dispatching is not needed. Present with an empty value: `STS$K_WARN`.
- `/HELP` is understood for a sub-verb. `__cli$show_verbs` lists a sub-verb table once and leaves the level as it found it.

The TLV routines

- `__tlv$put` checks the free space against the real length of the record, rejects a tag id above 4095 and a type above 15, and fails (nothing written) where it used to overrun. A UUID record adds its real length to the PDU. `valsz` for a word, longword, quadword or UUID must be at least the size of the type; **zero is refused (check)**.
- `__tlv$hset` rejects a signature over eight octets (it cut it) and NULL. `__tlv$hget` writes none of its outputs on a mismatch (it stored `STS$K_ERROR` in `*csr`).
- `__tlv$get` checks the wire length of a fixed-size record and reports a truncated one as an error; `__tlv$get` and `__tlv$lookup` report a record that runs past the PDU. `__tlv$dump` prints a value in chunks of 64 octets and no longer overruns its buffer.

Tests and documents

- Seven test programs are new: `fao_edge`, `tlv_hostile`, `cli_rules`, `queue`, `strings`, `params`, `log_misc`. `test_pattern_match` compares against a dynamic-programming reference. `tests/run.sh` runs all of them.
- The authorship line and the initials in the histories are corrected. The headers of `__util$movc5` and of the pattern matcher are written afresh. The module headers of the sources are shortened to the essentials.